

Services (Local)			
SQL Server Browser			
Stop the service Pause the service Restart the service			
Name	Status	Startup Type	
SQL Server Agent (SQLEXPRESS)		Disabled	
SQL Server Browser	Running	Automatic	
SQL Server CEIP service (MSSQLSERVER)	Running	Automatic	
SQL Server CEIP service (SQLEXPRESS)	Running	Automatic	

Next steps

Use a lightweight database to store data on the users device

See [Use a SQLite database in a UWP app](#).

Share code between different apps across multiple platforms

See [Share code between desktop and UWP](#).

Add master detail pages with Azure SQL back ends

See [Customer Orders Database sample](#).

Feedback

Was this page helpful?

☐ Yes

☐ No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

Data binding

Article • 10/20/2022

Data binding is a way for your app's UI to display data, and optionally to stay in sync with that data. Data binding allows you to separate the concern of data from the concern of UI, and that results in a simpler conceptual model as well as better readability, testability, and maintainability of your app. In markup, you can choose to use either the [{x:Bind} markup extension](#) or the [{Binding} markup extension](#). And you can even use a mixture of the two in the same app—even on the same UI element. {x:Bind} is new for Windows 10 and it has better performance.

Topic	Description
Data binding overview	This topic shows you how to bind a control (or other UI element) to a single item or bind an items control to a collection of items in a Universal Windows Platform (UWP) app. In addition, we show how to control the rendering of items, implement a details view based on a selection, and convert data for display. For more detailed info, see Data binding in depth .
Data binding in depth	This topic describes data binding features in detail.
Sample data on the design surface, and for prototyping	In order to have your controls populated with data in the Visual Studio designer (so that you can work on your app's layout, templates, and other visual properties), there are various ways in which you can use design-time sample data. Sample data can also be really useful and time-saving if you're building a sketch (or prototype) app. You can use sample data in your sketch or prototype at run-time to illustrate your ideas without going as far as connecting to real, live data.
Bind hierarchical data and create a master/details view	You can make a multi-level master/details (also known as list-details) view of hierarchical data by binding items controls to CollectionViewSource instances that are bound together in a chain.
Data binding and MVVM	This topic describes the Model-View-ViewModel (MVVM) UI architectural design pattern. Data binding is at the core of MVVM, and enables loose coupling between UI and non-UI code.

Data binding overview

Article • 10/20/2022

This topic shows you how to bind a control (or other UI element) to a single item or bind an items control to a collection of items in a Universal Windows Platform (UWP) app. In addition, we show how to control the rendering of items, implement a details view based on a selection, and convert data for display. For more detailed info, see [Data binding in depth](#).

Prerequisites

This topic assumes that you know how to create a basic UWP app. For instructions on creating your first UWP app, see [Get started with Windows apps](#).

Create the project

Create a new **Blank Application (Windows Universal)** project. Name it "Quickstart".

Binding to a single item

Every binding consists of a binding target and a binding source. Typically, the target is a property of a control or other UI element, and the source is a property of a class instance (a data model, or a view model). This example shows how to bind a control to a single item. The target is the **Text** property of a **TextBlock**. The source is an instance of a simple class named **Recording** that represents an audio recording. Let's look at the class first.

If you're using C# or C++/CX, then add a new class to your project, and name the class **Recording**.

If you're using C++/WinRT, then add new **Midl File (.idl)** items to the project, named as shown in the C++/WinRT code example listing below. Replace the contents of those new files with the [MIDL 3.0](#) code shown in the listing, build the project to generate `Recording.h` and `.cpp` and `RecordingViewModel.h` and `.cpp`, and then add code to the generated files to match the listing. For more info about those generated files and how to copy them into your project, see [XAML controls; bind to a C++/WinRT property](#).

```

namespace Quickstart
{
    public class Recording
    {
        public string ArtistName { get; set; }
        public string CompositionName { get; set; }
        public DateTime ReleaseDateTime { get; set; }
        public Recording()
        {
            this.ArtistName = "Wolfgang Amadeus Mozart";
            this.CompositionName = "Andante in C for Piano";
            this.ReleaseDateTime = new DateTime(1761, 1, 1);
        }
        public string OneLineSummary
        {
            get
            {
                return $"{this.CompositionName} by {this.ArtistName},
released: "
                    + this.ReleaseDateTime.ToString("d");
            }
        }
    }
    public class RecordingViewModel
    {
        private Recording defaultRecording = new Recording();
        public Recording DefaultRecording { get { return
this.defaultRecording; } }
    }
}

```

Next, expose the binding source class from the class that represents your page of markup. We do that by adding a property of type **RecordingViewModel** to **MainPage**.

If you're using **C++/WinRT**, then first update **MainPage.idl**. Build the project to regenerate **MainPage.h** and **.cpp**, and merge the changes in those generated files into the ones in your project.

C#

```

namespace Quickstart
{
    public sealed partial class MainPage : Page
    {
        public MainPage()
        {
            this.InitializeComponent();
            this.ViewModel = new RecordingViewModel();
        }
        public RecordingViewModel ViewModel { get; set; }
    }
}

```

```
}  
}
```

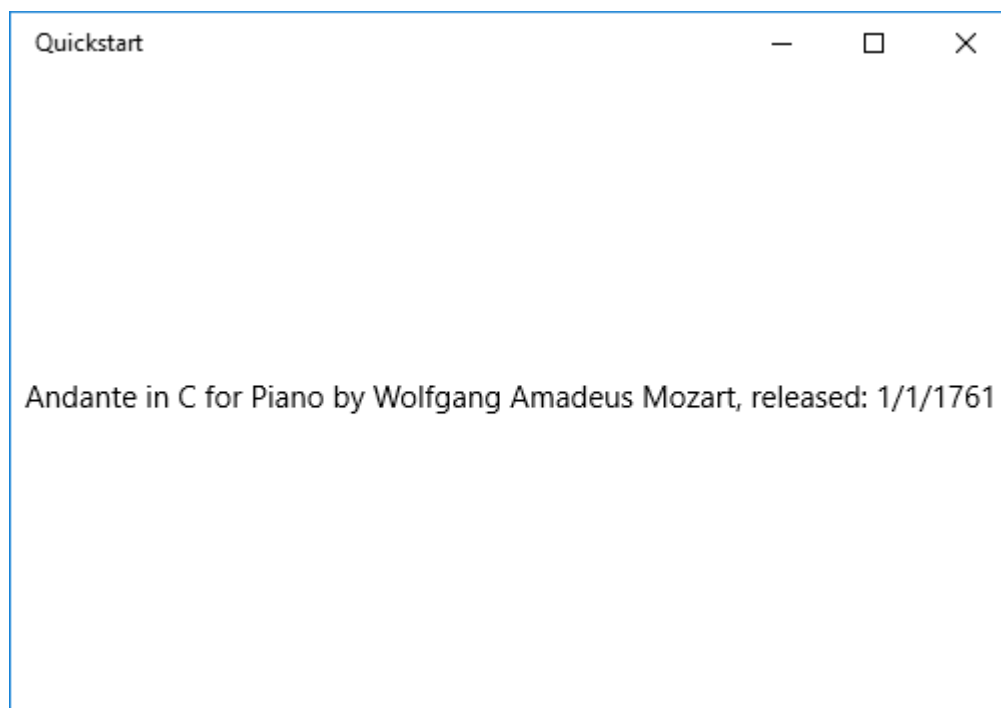
The last piece is to bind a **TextBlock** to the **ViewModel.DefaultRecording.OneLineSummary** property.

XML

```
<Page x:Class="Quickstart.MainPage" ... >  
    <Grid Background="{ThemeResource ApplicationPageBackgroundThemeBrush}">  
        <TextBlock Text="{x:Bind ViewModel.DefaultRecording.OneLineSummary}"  
            HorizontalAlignment="Center"  
            VerticalAlignment="Center"/>  
    </Grid>  
</Page>
```

If you're using [C++/WinRT](#), then you'll need to remove the **MainPage::ClickHandler** function in order for the project to build.

Here's the result.



Binding to a collection of items

A common scenario is to bind to a collection of business objects. In C# and Visual Basic, the generic **ObservableCollection<T>** class is a good collection choice for data binding, because it implements the **INotifyPropertyChanged** and **INotifyCollectionChanged** interfaces. These interfaces provide change notification to bindings when items are added or removed or a property of the list itself changes. If you want your bound

controls to update with changes to properties of objects in the collection, the business object should also implement **INotifyPropertyChanged**. For more info, see [Data binding in depth](#).

If you're using [C++/WinRT](#), then you can learn more about binding to an observable collection in [XAML items controls; bind to a C++/WinRT collection](#). If you read that topic first, then the intent of the C++/WinRT code listing shown below will be clearer.

This next example binds a [ListView](#) to a collection of `Recording` objects. Let's start by adding the collection to our view model. Just add these new members to the **RecordingViewModel** class.

C#

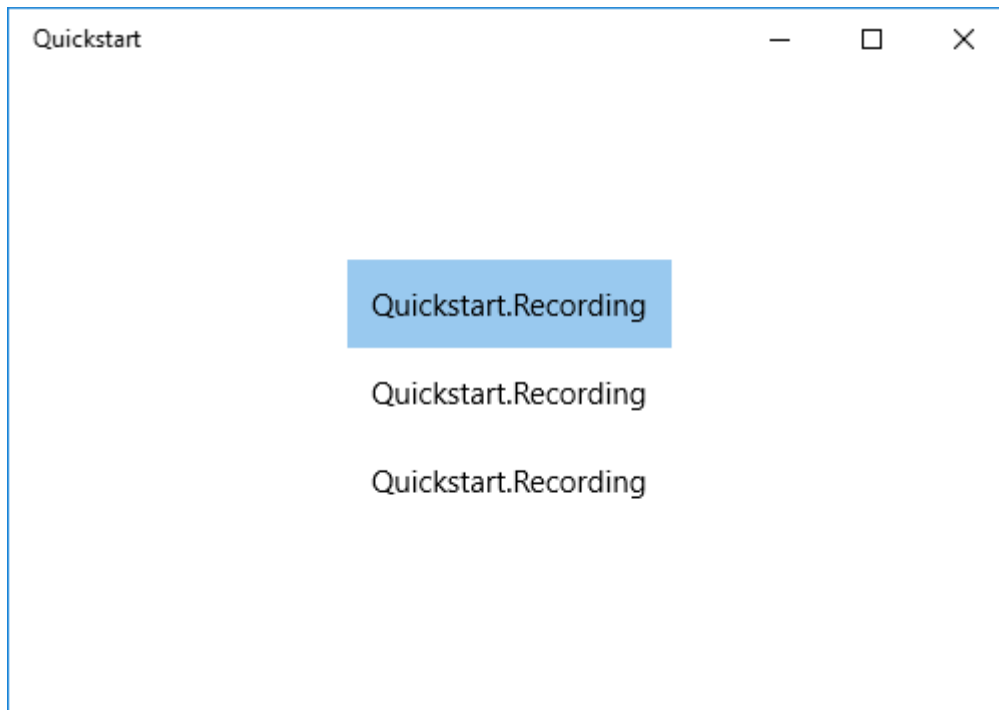
```
public class RecordingViewModel
{
    ...
    private ObservableCollection<Recording> recordings = new
    ObservableCollection<Recording>();
    public ObservableCollection<Recording> Recordings{ get{ return
    this.recordings; } }
    public RecordingViewModel()
    {
        this.recordings.Add(new Recording(){ ArtistName = "Johann Sebastian
        Bach",
            CompositionName = "Mass in B minor", ReleaseDateTime = new
            DateTime(1748, 7, 8) });
        this.recordings.Add(new Recording(){ ArtistName = "Ludwig van
        Beethoven",
            CompositionName = "Third Symphony", ReleaseDateTime = new
            DateTime(1805, 2, 11) });
        this.recordings.Add(new Recording(){ ArtistName = "George Frideric
        Handel",
            CompositionName = "Serse", ReleaseDateTime = new DateTime(1737,
            12, 3) });
    }
}
```

And then bind a [ListView](#) to the **ViewModel.Recordings** property.

XML

```
<Page x:Class="Quickstart.MainPage" ... >
    <Grid Background="{ThemeResource ApplicationPageBackgroundThemeBrush}">
        <ListView ItemsSource="{x:Bind ViewModel.Recordings}"
            HorizontalAlignment="Center" VerticalAlignment="Center"/>
    </Grid>
</Page>
```

We haven't yet provided a data template for the **Recording** class, so the best the UI framework can do is to call **ToString** for each item in the **ListView**. The default implementation of **ToString** is to return the type name.



To remedy this, we can either override **ToString** to return the value of **OneLineSummary**, or we can provide a data template. The data template option is a more usual solution, and a more flexible one. You specify a data template by using the **ContentTemplate** property of a content control or the **ItemTemplate** property of an items control. Here are two ways we could design a data template for **Recording** together with an illustration of the result.

XML

```
<ListView ItemsSource="{x:Bind ViewModel.Recordings}"
HorizontalAlignment="Center" VerticalAlignment="Center">
  <ListView.ItemTemplate>
    <DataTemplate x:DataType="local:Recording">
      <TextBlock Text="{x:Bind OneLineSummary}"/>
    </DataTemplate>
  </ListView.ItemTemplate>
</ListView>
```

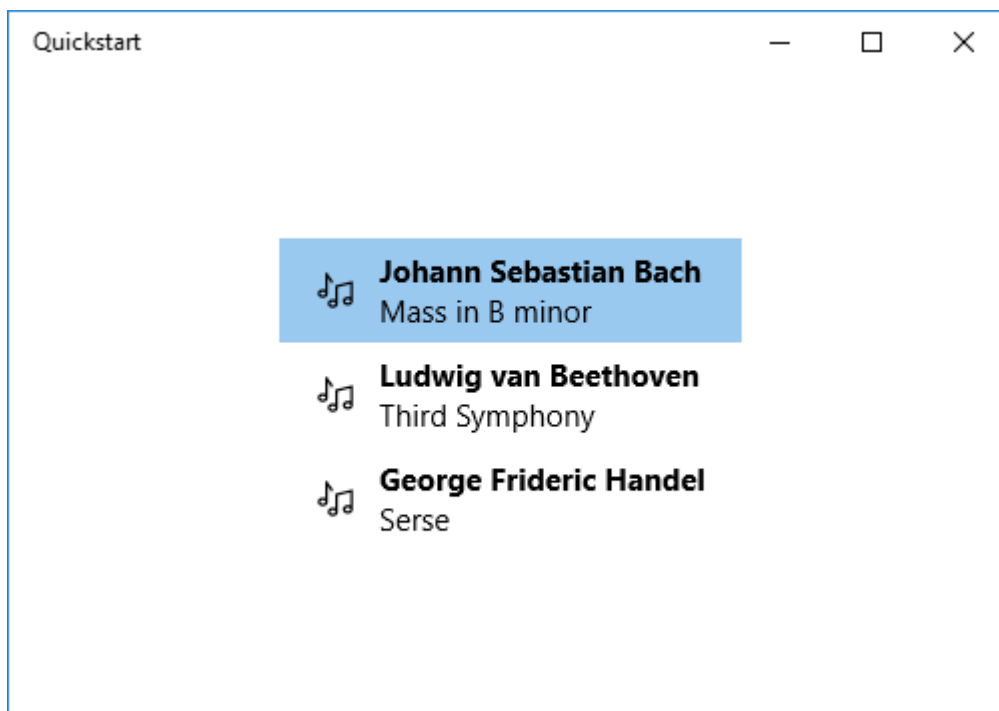
Mass in B minor by Johann Sebastian Bach, released: 7/8/1748

Third Symphony by Ludwig van Beethoven, released: 2/11/1805

Serse by George Frideric Handel, released: 12/3/1737

XML

```
<ListView ItemsSource="{x:Bind ViewModel.Recordings}"
HorizontalAlignment="Center" VerticalAlignment="Center">
  <ListView.ItemTemplate>
    <DataTemplate x:DataType="local:Recording">
      <StackPanel Orientation="Horizontal" Margin="6">
        <SymbolIcon Symbol="Audio" Margin="0,0,12,0"/>
        <StackPanel>
          <TextBlock Text="{x:Bind ArtistName}"
FontWeight="Bold"/>
          <TextBlock Text="{x:Bind CompositionName}"/>
        </StackPanel>
      </StackPanel>
    </DataTemplate>
  </ListView.ItemTemplate>
</ListView>
```

For more information about XAML syntax, see [Create a UI with XAML](#). For more information about control layout, see [Define layouts with XAML](#).

Adding a details view

You can choose to display all the details of **Recording** objects in **ListView** items. But that takes up a lot of space. Instead, you can show just enough data in the item to identify it and then, when the user makes a selection, you can display all the details of the selected item in a separate piece of UI known as the details view. This arrangement is also known as a master/details view, or a list/details view.

There are two ways to go about this. You can bind the details view to the **SelectedItem** property of the **ListView**. Or you can use a **CollectionViewSource**, in which case you bind both the **ListView** and the details view to the **CollectionViewSource** (doing so takes care of the currently-selected item for you). Both techniques are shown below, and they both give the same results (shown in the illustration).

ⓘ Note

So far in this topic we've only used the **{x:Bind}** markup extension, but both of the techniques we'll show below require the more flexible (but less performant) **{Binding}** markup extension.

If you're using C++/WinRT or Visual C++ component extensions (C++/CX) then, to use the **{Binding}** markup extension, you'll need to add the **BindableAttribute** attribute to any runtime class that you want to bind to. To use **{x:Bind}**, you don't need that attribute.

Important

If you're using **C++/WinRT**, then the **BindableAttribute** attribute is available if you've installed the Windows SDK version 10.0.17763.0 (Windows 10, version 1809), or later. Without that attribute, you'll need to implement the **ICustomPropertyProvider** and **ICustomProperty** interfaces in order to be able to use the **{Binding}** markup extension.

First, here's the **SelectedItem** technique.

C#

```
// No code changes necessary for C#.
```

The only other change necessary is to the markup.

XML

```
<Page x:Class="Quickstart.MainPage" ... >
    <Grid Background="{ThemeResource ApplicationPageBackgroundThemeBrush}">
        <StackPanel HorizontalAlignment="Center" VerticalAlignment="Center">
            <ListView x:Name="recordingsListView" ItemsSource="{x:Bind
ViewModel.Recordings}">
                <ListView.ItemTemplate>
                    <DataTemplate x:DataType="local:Recording">
                        <StackPanel Orientation="Horizontal" Margin="6">
                            <SymbolIcon Symbol="Audio" Margin="0,0,12,0"/>
                            <StackPanel>
                                <TextBlock Text="{x:Bind CompositionName}"/>
                            </StackPanel>
                        </StackPanel>
                    </DataTemplate>
                </ListView.ItemTemplate>
            </ListView>
            <StackPanel DataContext="{Binding SelectedItem,
ElementName=recordingsListView}"
                Margin="0,24,0,0">
                <TextBlock Text="{Binding ArtistName}"/>
                <TextBlock Text="{Binding CompositionName}"/>
                <TextBlock Text="{Binding ReleaseDateTime}"/>
            </StackPanel>
        </StackPanel>
    </Grid>
</Page>
```

For the **CollectionViewSource** technique, first add a **CollectionViewSource** as a page resource.

XML

```
<Page.Resources>
    <CollectionViewSource x:Name="RecordingsCollection" Source="{x:Bind
ViewModel.Recordings}"/>
</Page.Resources>
```

And then adjust the bindings on the **ListView** (which no longer needs to be named) and on the details view to use the **CollectionViewSource**. Note that by binding the details view directly to the **CollectionViewSource**, you're implying that you want to bind to the current item in bindings where the path cannot be found on the collection itself. There's no need to specify the **CurrentItem** property as the path for the binding, although you can do that if there's any ambiguity).

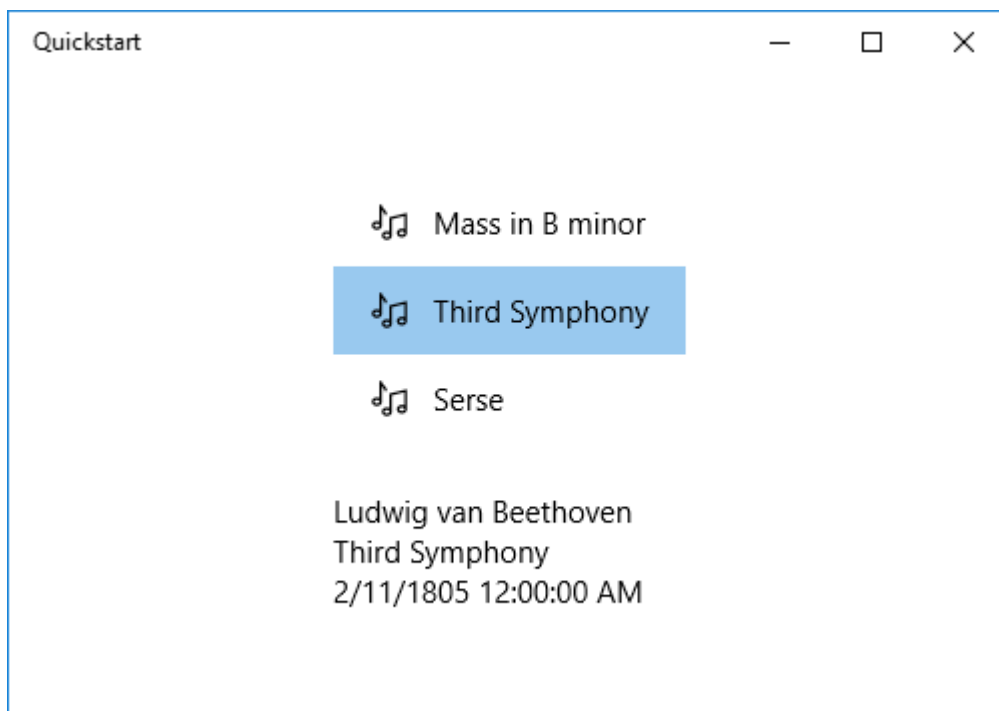
XML

```
...
<ListView ItemsSource="{Binding Source={StaticResource
RecordingsCollection}}">
...
<StackPanel DataContext="{Binding Source={StaticResource
RecordingsCollection}}" ...>
...
```

And here's the identical result in each case.

ⓘ Note

If you're using C++, then your UI won't look exactly like the illustration below: the rendering of the **ReleaseDateTime** property is different. See the following section for more discussion of this.



Formatting or converting data values for display

There is an issue with the rendering above. The **ReleaseDateTime** property is not just a date, it's a **DateTime** (if you're using C++, then it's a **Calendar**). So, in C#, it's being displayed with more precision than we need. And in C++ it's being rendered as a type name. One solution is to add a string property to the **Recording** class that returns the equivalent of `this.ReleaseDateTime.ToString("d")`. Naming that property **ReleaseDate** would indicate that it returns a date, and not a date-and-time. Naming it **ReleaseDateAsString** would further indicate that it returns a string.

A more flexible solution is to use something known as a value converter. Here's an example of how to author your own value converter. If you're using C#, then add the code below to your `Recording.cs` source code file. If you're using C++/WinRT, then add a new **Midl File (.idl)** item to the project, named as shown in the C++/WinRT code example listing below, build the project to generate `StringFormatter.h` and `.cpp`, add those files to your project, and then paste the code listings into them. Also add `#include "StringFormatter.h"` to `MainPage.h`.

C#

```
public class StringFormatter : Windows.UI.Xaml.Data.IValueConverter
{
    // This converts the value object to the string to display.
    // This will work with most simple types.
    public object Convert(object value, Type targetType,
        object parameter, string language)
```

```

{
    // Retrieve the format string and use it to format the value.
    string formatString = parameter as string;
    if (!string.IsNullOrEmpty(formatString))
    {
        return string.Format(formatString, value);
    }

    // If the format string is null or empty, simply
    // call ToString() on the value.
    return value.ToString();
}

// No need to implement converting back on a one-way binding
public object ConvertBack(object value, Type targetType,
    object parameter, string language)
{
    throw new NotImplementedException();
}
}

```

ⓘ Note

For the C++/WinRT code listing above, in `StringFormatter.idl`, we use the **default attribute** to declare **IValueConverter** as the default interface. In the listing, **StringFormatter** has only a constructor, and no methods, so no default interface is generated for it. The `default` attribute is optimal if you won't be adding instance members to **StringFormatter**, because no `QueryInterface` will be required to call the **IValueConverter** methods. Alternatively, you can prompt a default **IStringFormatter** interface to be generated, and you do that by annotating the runtime class itself with the **default_interface attribute**. That option is optimal if you add instance members to **StringFormatter** that are called more often than the methods of **IValueConverter** are, because then no `QueryInterface` will be required to call the instance members.

Now we can add an instance of **StringFormatter** as a page resource and use it in the binding of the **TextBlock** that displays the **ReleaseDateTime** property.

XML

```

<Page.Resources>
    <local:StringFormatter x:Key="StringFormatterValueConverter"/>
</Page.Resources>
...
<TextBlock Text="{Binding ReleaseDateTime,
    Converter={StaticResource StringFormatterValueConverter}},

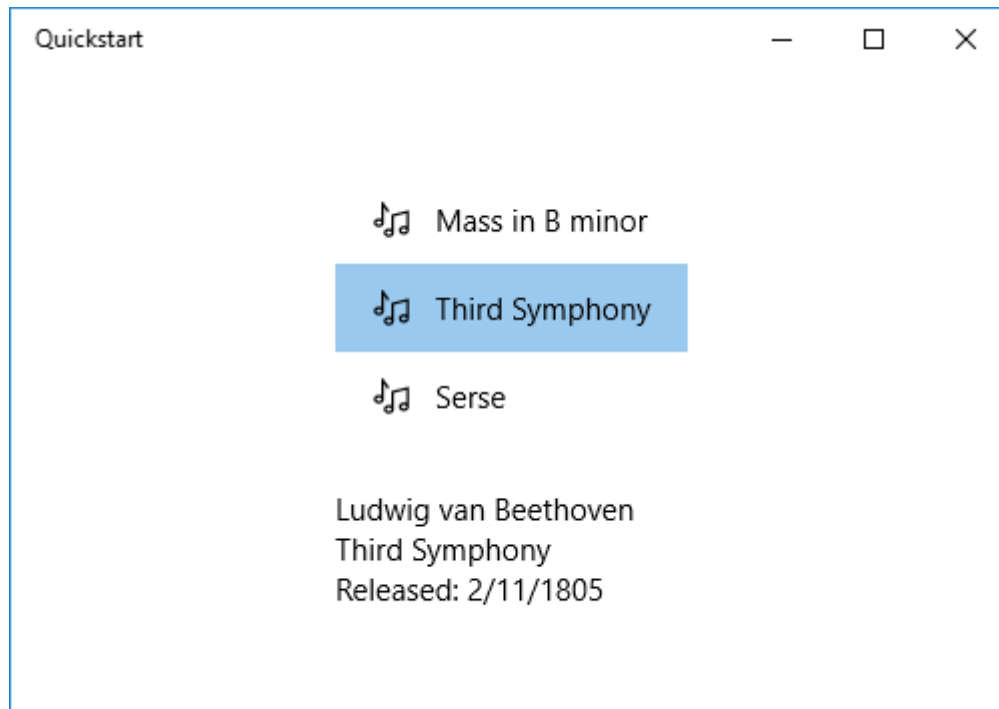
```

```
ConverterParameter=Released: \"{0:d}\"/>
```

...

As you can see above, for formatting flexibility we use the markup to pass a format string into the converter by way of the converter parameter. In the code examples shown in this topic, only the C# value converter makes use of that parameter. But you could easily pass a C++-style format string as the converter parameter, and use that in your value converter with a formatting function such as **wprintf** or **swprintf**.

Here's the result.



ⓘ Note

Starting in Windows 10, version 1607, the XAML framework provides a built-in Boolean-to-Visibility converter. The converter maps **true** to the **Visibility.Visible** enumeration value and **false** to **Visibility.Collapsed** so you can bind a Visibility property to a Boolean without creating a converter. To use the built in converter, your app's minimum target SDK version must be 14393 or later. You can't use it when your app targets earlier versions of Windows 10. For more info about target versions, see [Version-adaptive code](#).

See also

- [Data binding](#)

Data binding in depth

Article • 01/04/2023

Important APIs

- [{x:Bind} markup extension](#)
- [Binding class](#)
- [DataContext](#)
- [INotifyPropertyChanged](#)

ⓘ Note

This topic describes data binding features in detail. For a short, practical introduction, see [Data binding overview](#).

This topic is about data binding for the APIs that reside in the [Windows.UI.Xaml.Data](#) namespace.

Data binding is a way for your app's UI to display data, and optionally to stay in sync with that data. Data binding allows you to separate the concern of data from the concern of UI, and that results in a simpler conceptual model as well as better readability, testability, and maintainability of your app.

You can use data binding to simply display values from a data source when the UI is first shown, but not to respond to changes in those values. This is a mode of binding called *one-time*, and it works well for a value that doesn't change during run-time. Alternatively, you can choose to "observe" the values and to update the UI when they change. This mode is called *one-way*, and it works well for read-only data. Ultimately, you can choose to both observe and update, so that changes that the user makes to values in the UI are automatically pushed back into the data source. This mode is called *two-way*, and it works well for read-write data. Here are some examples.

- You could use the one-time mode to bind an [Image](#) to the current user's photo.
- You could use the one-way mode to bind a [ListView](#) to a collection of real-time news articles grouped by newspaper section.
- You could use the two-way mode to bind a [TextBox](#) to a customer's name in a form.

Independent of mode, there are two kinds of binding, and they're both typically declared in UI markup. You can choose to use either the [{x:Bind} markup extension](#) or the [{Binding} markup extension](#). And you can even use a mixture of the two in the same app—even on the same UI element. {x:Bind} is new for Windows 10 and it has better performance. All the details described in this topic apply to both kinds of binding unless we explicitly say otherwise.

Sample apps that demonstrate {x:Bind}

- [{x:Bind} sample](#) [↗](#).
- [QuizGame](#) [↗](#).
- [XAML UI Basics sample](#) [↗](#).

Sample apps that demonstrate {Binding}

- Download the [Bookstore1](#) [↗](#) app.
- Download the [Bookstore2](#) [↗](#) app.

Every binding involves these pieces

- A *binding source*. This is the source of the data for the binding, and it can be an instance of any class that has members whose values you want to display in your UI.
- A *binding target*. This is a [DependencyProperty](#) of the [FrameworkElement](#) in your UI that displays the data.
- A *binding object*. This is the piece that transfers data values from the source to the target, and optionally from the target back to the source. The binding object is created at XAML load time from your `{x:Bind}` or `{Binding}` markup extension.

In the following sections, we'll take a closer look at the binding source, the binding target, and the binding object. And we'll link the sections together with the example of binding a button's content to a string property named **NextButtonText**, which belongs to a class named **HostViewModel**.

Binding source

Here's a very rudimentary implementation of a class that we could use as a binding source.

If you're using [C++/WinRT](#), then add new **Midl File (.idl)** items to the project, named as shown in the C++/WinRT code example listing below. Replace the contents of those new files with the [MIDL 3.0](#) code shown in the listing, build the project to generate `HostViewModel.h` and `.cpp`, and then add code to the generated files to match the listing. For more info about those generated files and how to copy them into your project, see [XAML controls; bind to a C++/WinRT property](#).

C#

```
public class HostViewModel
{
    public HostViewModel()
    {
        this.NextButtonText = "Next";
    }

    public string NextButtonText { get; set; }
}
```

That implementation of **HostViewModel**, and its property **NextButtonText**, are only appropriate for one-time binding. But one-way and two-way bindings are extremely common, and in those kinds of binding the UI automatically updates in response to changes in the data values of the binding source. In order for those kinds of binding to work correctly, you need to make your binding source "observable" to the binding object. So in our example, if we want to one-way or two-way bind to the **NextButtonText** property, then any changes that happen at run-time to the value of that property need to be made observable to the binding object.

One way of doing that is to derive the class that represents your binding source from [DependencyObject](#), and expose a data value through a [DependencyProperty](#). That's how a [FrameworkElement](#) becomes observable. **FrameworkElements** are good binding sources right out of the box.

A more lightweight way of making a class observable—and a necessary one for classes that already have a base class—is to implement [System.ComponentModel.INotifyPropertyChanged](#). This really just involves implementing a single event named **PropertyChanged**. An example using **HostViewModel** is below.

C#

```
...
using System.ComponentModel;
using System.Runtime.CompilerServices;
...
public class HostViewModel : INotifyPropertyChanged
{
    private string nextButtonText;

    public event PropertyChangedEventHandler PropertyChanged = delegate { };

    public HostViewModel()
    {
        this.NextButtonText = "Next";
    }

    public string NextButtonText
    {
        get { return this.nextButtonText; }
        set
        {
            this.nextButtonText = value;
            this.OnPropertyChanged();
        }
    }

    public void OnPropertyChanged([CallerMemberName] string propertyName = null)
    {
        // Raise the PropertyChanged event, passing the name of the property whose value has
        changed.
        this.PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }
}
```

Now the **NextButtonText** property is observable. When you author a one-way or a two-way binding to that property (we'll show how later), the resulting binding object subscribes to the **PropertyChanged** event. When that event is raised, the binding object's handler receives an argument containing the name of the property that has changed. That's how the binding object knows which property's value to go and read again.

So that you don't have to implement the pattern shown above multiple times, if you're using C# then you can just derive from the **BindableBase** base class that you'll find in the [QuizGame](#) sample (in the "Common" folder). Here's an example of how that looks.

C#

```
public class HostViewModel : BindableBase
{
    private string nextButtonText;

    public HostViewModel()
    {
        this.NextButtonText = "Next";
    }

    public string NextButtonText
    {
        get { return this.nextButtonText; }
        set { this.SetProperty(ref this.nextButtonText, value); }
    }
}
```

```
}  
}
```

❗ Note

For C++/WinRT, any runtime class that you declare in your application that derives from a base class is known as a *composable* class. And there are constraints around composable classes. For an application to pass the **Windows App Certification Kit** tests used by Visual Studio and by the Microsoft Store to validate submissions (and therefore for the application to be successfully ingested into the Microsoft Store), a composable class must ultimately derive from a Windows base class. Meaning that the class at the very root of the inheritance hierarchy must be a type originating in a *Windows.** namespace. If you do need to derive a runtime class from a base class—for example, to implement a **BindableBase** class for all of your view models to derive from—then you can derive from **Windows.UI.Xaml.DependencyObject**.

Raising the **PropertyChanged** event with an argument of **String.Empty** or **null** indicates that all non-indexer properties on the object should be re-read. You can raise the event to indicate that indexer properties on the object have changed by using an argument of "Item[*indexer*]" for specific indexers (where *indexer* is the index value), or a value of "Item[]" for all indexers.

A binding source can be treated either as a single object whose properties contain data, or as a collection of objects. In C# and Visual Basic code, you can one-time bind to an object that implements **List(Of T)** to display a collection that doesn't change at run-time. For an observable collection (observing when items are added to and removed from the collection), one-way bind to **ObservableCollection(Of T)** instead. In C++/CX code, you can bind to **Vector<T>** for both observable and non-observable collections, and C++/WinRT has its own types. To bind to your own collection classes, use the guidance in the following table.

Scenario	C# and VB (CLR)	C++/WinRT	C++/CX
Bind to an object.	Can be any object.	Can be any object.	Object must have BindableAttribute or implement ICustomPropertyProvider .
Get property change notifications from a bound object.	Object must implement INotifyPropertyChanged .	Object must implement INotifyPropertyChanged .	Object must implement INotifyPropertyChanged .
Bind to a collection.	List(Of T)	IVector of IInspectable , or IBindableObservableVector . See XAML items controls ; bind to a C++/WinRT collection and Collections with C++/WinRT.	Vector<T>

Scenario	C# and VB (CLR)	C++/WinRT	C++/CX
Get collection change notifications from a bound collection.	ObservableCollection(Of T)	IObservableVector of IInspectable . For example, winrt::single_threaded_observable_vector<T> .	IObservableVector<T> . Vector<T> implements this interface.
Implement a collection that supports binding.	Extend List(Of T) or implement IList , IList(Of Object) , IEnumerable , or IEnumerable(Of Object) . Binding to generic IList(Of T) and IEnumerable(Of T) is not supported.	Implement IVector of IInspectable . See XAML items controls ; bind to a C++/WinRT collection and Collections with C++/WinRT .	Implement IBindableVector , IBindableIterable , IVector<Object^> , IIterable<Object^> , IVector<IInspectable*> , or IIterable<IInspectable*> . Binding to generic IVector<T> and IIterable<T> is not supported.
Implement a collection that supports collection change notifications.	Extend ObservableCollection(Of T) or implement (non-generic) IList and INotifyCollectionChanged .	Implement IObservableVector of IInspectable , or IBindableObservableVector .	Implement IBindableVector and IBindableObservableVector .
Implement a collection that supports incremental loading.	Extend ObservableCollection(Of T) or implement (non-generic) IList and INotifyCollectionChanged . Additionally, implement ISupportIncrementalLoading .	Implement IObservableVector of IInspectable , or IBindableObservableVector . Additionally, implement ISupportIncrementalLoading	Implement IBindableVector , IBindableObservableVector , and ISupportIncrementalLoading .

You can bind list controls to arbitrarily large data sources, and still achieve high performance, by using incremental loading. For example, you can bind list controls to Bing image query results without having to load all the results at once. Instead, you load only some results immediately, and load additional results as needed. To support incremental loading, you must implement [ISupportIncrementalLoading](#) on a data source that supports collection change notifications. When the data binding engine requests more data, your data source must make the appropriate requests, integrate the results, and then send the appropriate notifications in order to update the UI.

Binding target

In the two examples below, the **Button.Content** property is the binding target, and its value is set to a markup extension that declares the binding object. First `{x:Bind}` is shown, and then `{Binding}`. Declaring bindings in markup is the common case (it's convenient, readable, and toolable). But you can avoid markup and imperatively (programmatically) create an instance of the [Binding](#) class instead if you need to.

XAML

```
<Button Content="{x:Bind ...}" ... />
```

XAML

```
<Button Content="{Binding ...}" ... />
```

If you're using C++/WinRT or Visual C++ component extensions (C++/CX), then you'll need to add the **BindableAttribute** attribute to any runtime class that you want to use the **{Binding}** markup extension with.

❗ Important

If you're using **C++/WinRT**, then the **BindableAttribute** attribute is available if you've installed the Windows SDK version 10.0.17763.0 (Windows 10, version 1809), or later. Without that attribute, you'll need to implement the **ICustomPropertyProvider** and **ICustomProperty** interfaces in order to be able to use the **{Binding}** markup extension.

Binding object declared using {x:Bind}

There's one step we need to do before we author our **{x:Bind}** markup. We need to expose our binding source class from the class that represents our page of markup. We do that by adding a property (of type **HostViewModel** in this case) to our **MainPage** page class.

C#

```
namespace DataBindingInDepth
{
    public sealed partial class MainPage : Page
    {
        public MainPage()
        {
            this.InitializeComponent();
            this.ViewModel = new HostViewModel();
        }

        public HostViewModel ViewModel { get; set; }
    }
}
```

That done, we can now take a closer look at the markup that declares the binding object. The example below uses the same **Button.Content** binding target we used in the "Binding target" section earlier, and shows it being bound to the **HostViewModel.NextButtonText** property.

XAML

```
<!-- MainPage.xaml -->
<Page x:Class="DataBindingInDepth.Mainpage" ... >
    <Button Content="{x:Bind Path=ViewModel.NextButtonText, Mode=OneWay}" ... />
</Page>
```

Notice the value that we specify for **Path**. This value is interpreted in the context of the page itself, and in this case the path begins by referencing the **ViewModel** property that we just added to the **MainPage** page. That property returns a **HostViewModel** instance, and so we can dot into that object to access the **HostViewModel.NextButtonText** property. And we specify **Mode**, to override the **{x:Bind}** default of one-time.

The **Path** property supports a variety of syntax options for binding to nested properties, attached properties, and integer and string indexers. For more info, see [Property-path syntax](#). Binding to string indexers gives you the effect of binding to dynamic properties without having to implement **ICustomPropertyProvider**. For other settings, see [{x:Bind} markup extension](#).

To illustrate that the **HostViewModel.NextButtonText** property is indeed observable, add a **Click** event handler to the button, and update the value of **HostViewModel.NextButtonText**. Build, run, and click the button to see the value of the button's **Content** update.

C#

```
// MainPage.xaml.cs
private void Button_Click(object sender, RoutedEventArgs e)
{
    this.ViewModel.NextButtonText = "Updated Next button text";
}
```

ⓘ Note

Changes to **TextBox.Text** are sent to a two-way bound source when the **TextBox** loses focus, and not after every user keystroke.

DataTemplate and x:DataType

Inside a **DataTemplate** (whether used as an item template, a content template, or a header template), the value of **Path** is not interpreted in the context of the page, but in the context of the data object being templated. When using **{x:Bind}** in a data template, so that its bindings can be validated (and efficient code generated for them) at compile-time, the **DataTemplate** needs to declare the type of its data object using **x:DataType**. The example given below could be used as the **ItemTemplate** of an items control bound to a collection of **SampleDataGroup** objects.

XAML

```
<DataTemplate x:Key="SimpleItemTemplate" x:DataType="data:SampleDataGroup">
    <StackPanel Orientation="Vertical" Height="50">
        <TextBlock Text="{x:Bind Title}"/>
        <TextBlock Text="{x:Bind Description}"/>
    </StackPanel>
</DataTemplate>
```

Weakly-typed objects in your Path

Consider for example that you have a type named **SampleDataGroup**, which implements a string property named **Title**. And you have a property **MainPage.SampleDataGroupAsObject**, which is of type **object**, but which actually returns an instance of **SampleDataGroup**. The binding `<TextBlock Text="{x:Bind SampleDataGroupAsObject.Title}"/>` will result in a compile error because the **Title** property is not found on the type **object**. The remedy for this is to add a cast to your **Path** syntax like this: `<TextBlock Text="{x:Bind ((data:SampleDataGroup)SampleDataGroupAsObject).Title}"/>`. Here's another example where **Element** is declared as **object** but is actually a **TextBlock**: `<TextBlock Text="{x:Bind Element.Text}"/>`. And a cast remedies the issue: `<TextBlock Text="{x:Bind ((TextBlock)Element).Text}"/>`.

If your data loads asynchronously

Code to support `{x:Bind}` is generated at compile-time in the partial classes for your pages. These files can be found in your `obj` folder, with names like (for C#) `<view name>.g.cs`. The generated code includes a handler for your page's **Loading** event, and that handler calls the **Initialize** method on a generated class that represents your page's bindings. **Initialize** in turn calls **Update** to begin moving data between the binding source and the target. **Loading** is raised just before the first measure pass of the page or user control. So if your data is loaded asynchronously it may not be ready by the time **Initialize** is called. So, after you've loaded data, you can force one-time bindings to be initialized by calling `this.Bindings.Update();`. If you only need one-time bindings for asynchronously-loaded data then it's much cheaper to initialize them this way than it is to have one-way bindings and to listen for changes. If your data does not undergo fine-grained changes, and if it's likely to be updated as part of a specific action, then you can make your bindings one-time, and force a manual update at any time with a call to **Update**.

❗ Note

`{x:Bind}` is not suited to late-bound scenarios, such as navigating the dictionary structure of a JSON object, nor duck typing. "Duck typing" is a weak form of typing based on lexical matches on property names (as in, "if it walks, swims, and quacks like a duck, then it's a duck"). With duck typing, a binding to the **Age** property would be equally satisfied with a **Person** or a **Wine** object (assuming that those types each had an **Age** property). For these scenarios, use the `{Binding}` markup extension.

Binding object declared using `{Binding}`

If you're using C++/WinRT or Visual C++ component extensions (C++/CX) then, to use the `{Binding}` markup extension, you'll need to add the `BindableAttribute` attribute to any runtime class that you want to bind to. To use `{x:Bind}`, you don't need that attribute.

C++/WinRT

```
// HostViewModel.idl
// Add this attribute:
[Windows.UI.Xaml.Data.Bindable]
runtimeclass HostViewModel : Windows.UI.Xaml.Data.INotifyPropertyChanged
...
```

❗ Important

If you're using **C++/WinRT**, then the `BindableAttribute` attribute is available if you've installed the Windows SDK version 10.0.17763.0 (Windows 10, version 1809), or later. Without that attribute, you'll need to implement the `ICustomPropertyProvider` and `ICustomProperty` interfaces in order to be able to use the `{Binding}` markup extension.

`{Binding}` assumes, by default, that you're binding to the `DataContext` of your markup page. So we'll set the `DataContext` of our page to be an instance of our binding source class (of type `HostViewModel` in this case). The example below shows the markup that declares the binding object. We use the same `Button.Content` binding target we used in the "Binding target" section earlier, and we bind to the `HostViewModel.NextButtonText` property.

XAML

```

<Page xmlns:viewmodel="using:DataBindingInDepth" ... >
    <Page.DataContext>
        <viewmodel:HostViewModel x:Name="viewModelInDataContext"/>
    </Page.DataContext>
    ...
    <Button Content="{Binding Path=NextButtonText}" ... />
</Page>

```

C#

```

// MainPage.xaml.cs
private void Button_Click(object sender, RoutedEventArgs e)
{
    this.viewModelInDataContext.NextButtonText = "Updated Next button text";
}

```

Notice the value that we specify for **Path**. This value is interpreted in the context of the page's **DataContext**, which in this example is set to an instance of **HostViewModel**. The path references the **HostViewModel.NextButtonText** property. We can omit **Mode**, because the **{Binding}** default of one-way works here.

The default value of **DataContext** for a UI element is the inherited value of its parent. You can of course override that default by setting **DataContext** explicitly, which is in turn inherited by children by default. Setting **DataContext** explicitly on an element is useful when you want to have multiple bindings that use the same source.

A binding object has a **Source** property, which defaults to the **DataContext** of the UI element on which the binding is declared. You can override this default by setting **Source**, **RelativeSource**, or **ElementName** explicitly on the binding (see **{Binding}** for details).

Inside a **DataTemplate**, the **DataContext** is automatically set to the data object being templated. The example given below could be used as the **ItemTemplate** of an items control bound to a collection of any type that has string properties named **Title** and **Description**.

XAML

```

<DataTemplate x:Key="SimpleItemTemplate">
    <StackPanel Orientation="Vertical" Height="50">
        <TextBlock Text="{Binding Title}"/>
        <TextBlock Text="{Binding Description}"/>
    </StackPanel>
</DataTemplate>

```

ⓘ Note

By default, changes to **TextBox.Text** are sent to a two-way bound source when the **TextBox** loses focus. To cause changes to be sent after every user keystroke, set **UpdateSourceTrigger** to **PropertyChanged** on the binding in markup. You can also completely take control of when changes are sent to the source by setting **UpdateSourceTrigger** to **Explicit**. You then handle events on the text box (typically **TextBox.TextChanged**), call **GetBindingExpression** on the target to get a **BindingExpression** object, and finally call **BindingExpression.UpdateSource** to programmatically update the data source.

The [Path](#) property supports a variety of syntax options for binding to nested properties, attached properties, and integer and string indexers. For more info, see [Property-path syntax](#). Binding to string indexers gives you the effect of binding to dynamic properties without having to implement [ICustomPropertyProvider](#). The [ElementName](#) property is useful for element-to-element binding. The [RelativeSource](#) property has several uses, one of which is as a more powerful alternative to template binding inside a [ControlTemplate](#). For other settings, see [{Binding} markup extension](#) and the [Binding](#) class.

What if the source and the target are not the same type?

If you want to control the visibility of a UI element based on the value of a boolean property, or if you want to render a UI element with a color that's a function of a numeric value's range or trend, or if you want to display a date and/or time value in a UI element property that expects a string, then you'll need to convert values from one type to another. There will be cases where the right solution is to expose another property of the right type from your binding source class, and keep the conversion logic encapsulated and testable there. But that isn't flexible nor scalable when you have large numbers, or large combinations, of source and target properties. In that case you have a couple of options:

- If using {x:Bind} then you can bind directly to a function to do that conversion
- Or you can specify a value converter which is an object designed to perform the conversion

Value Converters

Here's a value converter, suitable for a one-time or a one-way binding, that converts a [DateTime](#) value to a string value containing the month. The class implements [IValueConverter](#).

C#

```
public class DateToStringConverter : IValueConverter
{
    // Define the Convert method to convert a DateTime value to
    // a month string.
    public object Convert(object value, Type targetType,
        object parameter, string language)
    {
        // value is the data from the source object.
        DateTime thisdate = (DateTime)value;
        int monthnum = thisdate.Month;
        string month;
        switch (monthnum)
        {
            case 1:
                month = "January";
                break;
            case 2:
                month = "February";
                break;
            default:
                month = "Month not found";
                break;
        }
        // Return the value to pass to the target.
        return month;
    }

    // ConvertBack is not implemented for a OneWay binding.
    public object ConvertBack(object value, Type targetType,
```



```

        object parameter, string language)
    {
        throw new NotImplementedException();
    }
}

```

And here's how you consume that value converter in your binding object markup.

XAML

```

<UserControl.Resources>
    <local:DateToStringConverter x:Key="Converter1"/>
</UserControl.Resources>
...
<TextBlock Grid.Column="0"
    Text="{x:Bind ViewModel.Month, Converter={StaticResource Converter1}}"/>
<TextBlock Grid.Column="0"
    Text="{Binding Month, Converter={StaticResource Converter1}}"/>

```

The binding engine calls the [Convert](#) and [ConvertBack](#) methods if the [Converter](#) parameter is defined for the binding. When data is passed from the source, the binding engine calls **Convert** and passes the returned data to the target. When data is passed from the target (for a two-way binding), the binding engine calls **ConvertBack** and passes the returned data to the source.

The converter also has optional parameters: [ConverterLanguage](#), which allows specifying the language to be used in the conversion, and [ConverterParameter](#), which allows passing a parameter for the conversion logic. For an example that uses a converter parameter, see [IValueConverter](#).

ⓘ Note

If there is an error in the conversion, do not throw an exception. Instead, return [DependencyProperty.UnsetValue](#), which will stop the data transfer.

To display a default value to use whenever the binding source cannot be resolved, set the **FallbackValue** property on the binding object in markup. This is useful to handle conversion and formatting errors. It is also useful to bind to source properties that might not exist on all objects in a bound collection of heterogeneous types.

If you bind a text control to a value that is not a string, the data binding engine will convert the value to a string. If the value is a reference type, the data binding engine will retrieve the string value by calling [ICustomPropertyProvider.GetStringRepresentation](#) or [IStringable.ToString](#) if available, and will otherwise call [Object.ToString](#). Note, however, that the binding engine will ignore any **ToString** implementation that hides the base-class implementation. Subclass implementations should override the base class **ToString** method instead. Similarly, in native languages, all managed objects appear to implement [ICustomPropertyProvider](#) and [IStringable](#). However, all calls to **GetStringRepresentation** and [IStringable.ToString](#) are routed to **Object.ToString** or an override of that method, and never to a new **ToString** implementation that hides the base-class implementation.

ⓘ Note

Starting in Windows 10, version 1607, the XAML framework provides a built in boolean to Visibility converter. The converter maps **true** to the **Visible** enumeration value and **false** to **Collapsed** so you can

bind a Visibility property to a boolean without creating a converter. To use the built in converter, your app's minimum target SDK version must be 14393 or later. You can't use it when your app targets earlier versions of Windows 10. For more info about target versions, see [Version adaptive code](#).

Function binding in {x:Bind}

{x:Bind} enables the final step in a binding path to be a function. This can be used to perform conversions, and to perform bindings that depend on more than one property. See [Functions in x:Bind](#)

Element-to-element binding

You can bind the property of one XAML element to the property of another XAML element. Here's an example of how that looks in markup.

XAML

```
<TextBox x:Name="myTextBox" />
<TextBlock Text="{x:Bind myTextBox.Text, Mode=OneWay}" />
```

Important

For the necessary workflow for element-to-element binding using C++/WinRT, see [Element-to-element binding](#).

Resource dictionaries with {x:Bind}

The {x:Bind} [markup extension](#) depends on code generation, so it needs a code-behind file containing a constructor that calls **InitializeComponent** (to initialize the generated code). You re-use the resource dictionary by instantiating its type (so that **InitializeComponent** is called) instead of referencing its filename. Here's an example of what to do if you have an existing resource dictionary and you want to use {x:Bind} in it.

TemplatesResourceDictionary.xaml

XAML

```
<ResourceDictionary
    x:Class="ExampleNamespace.TemplatesResourceDictionary"
    ....
    xmlns:exemplenamespace="using:ExampleNamespace">

    <DataTemplate x:Key="EmployeeTemplate" x:DataType="exemplenamespace:IEmployee">
        <Grid>
            <TextBlock Text="{x:Bind Name}" />
        </Grid>
    </DataTemplate>
</ResourceDictionary>
```

TemplatesResourceDictionary.xaml.cs

C#

```
using Windows.UI.Xaml.Data;

namespace ExampleNamespace
{
    public partial class TemplatesResourceDictionary
    {
        public TemplatesResourceDictionary()
        {
            InitializeComponent();
        }
    }
}
```

MainPage.xaml

XAML

```
<Page x:Class="ExampleNamespace.MainPage"
      ....
      xmlns:examplenamespace="using:ExampleNamespace">

    <Page.Resources>
        <ResourceDictionary>
            ....
            <ResourceDictionary.MergedDictionaries>
                <examplenamespace:TemplatesResourceDictionary/>
            </ResourceDictionary.MergedDictionaries>
        </ResourceDictionary>
    </Page.Resources>
</Page>
```

Event binding and ICommand

`{x:Bind}` supports a feature called event binding. With this feature, you can specify the handler for an event using a binding, which is an additional option on top of handling events with a method on the code-behind file. Let's say you have a **RootFrame** property on your **MainPage** class.

C#

```
public sealed partial class MainPage : Page
{
    ...
    public Frame RootFrame { get { return Window.Current.Content as Frame; } }
}
```

You can then bind a button's **Click** event to a method on the **Frame** object returned by the **RootFrame** property like this. Note that we also bind the button's **IsEnabled** property to another member of the same **Frame**.

XAML

```
<AppBarButton Icon="Forward" IsCompact="True"
              IsEnabled="{x:Bind RootFrame.CanGoForward, Mode=OneWay}"
              Click="{x:Bind RootFrame.GoForward}"/>
```

Overloaded methods cannot be used to handle an event with this technique. Also, if the method that handles the event has parameters then they must all be assignable from the types of all of the event's parameters, respectively. In this case, [Frame.GoForward](#) is not overloaded and it has no parameters (but it would still be valid even if it took two **object** parameters). [Frame.GoBack](#) is overloaded, though, so we can't use that method with this technique.

The event binding technique is similar to implementing and consuming commands (a command is a property that returns an object that implements the [ICommand](#) interface). Both `{x:Bind}` and `{Binding}` work with commands. So that you don't have to implement the command pattern multiple times, you can use the [DelegateCommand](#) helper class that you'll find in the [QuizGame](#) sample (in the "Common" folder).

Binding to a collection of folders or files

You can use the APIs in the [Windows.Storage](#) namespace to retrieve folder and file data. However, the various [GetFilesAsync](#), [GetFoldersAsync](#), and [GetItemsAsync](#) methods do not return values that are suitable for binding to list controls. Instead, you must bind to the return values of the [GetVirtualizedFilesVector](#), [GetVirtualizedFoldersVector](#), and [GetVirtualizedItemsVector](#) methods of the [FileInformationFactory](#) class. The following code example from the [StorageDataSource](#) and [GetVirtualizedFilesVector](#) sample shows the typical usage pattern. Remember to declare the **picturesLibrary** capability in your app package manifest, and confirm that there are pictures in your Pictures library folder.

C#

```
protected override void OnNavigatedTo(NavigationEventArgs e)
{
    var library = Windows.Storage.KnownFolders.PicturesLibrary;
    var queryOptions = new Windows.Storage.Search.QueryOptions();
    queryOptions.FolderDepth = Windows.Storage.Search.FolderDepth.Deep;
    queryOptions.IndexerOption = Windows.Storage.Search.IndexerOption.UseIndexerWhenAvailable;

    var fileQuery = library.CreateFileQueryWithOptions(queryOptions);

    var fif = new Windows.Storage.BulkAccess.FileInformationFactory(
        fileQuery,
        Windows.Storage.FileProperties.ThumbnailMode.PicturesView,
        190,
        Windows.Storage.FileProperties.ThumbnailOptions.UseCurrentScale,
        false
    );

    var dataSource = fif.GetVirtualizedFilesVector();
    this.PicturesListView.ItemsSource = dataSource;
}
```

You will typically use this approach to create a read-only view of file and folder info. You can create two-way bindings to the file and folder properties, for example to let users rate a song in a music view. However, any changes are not persisted until you call the appropriate [SavePropertiesAsync](#) method (for example, [MusicProperties.SavePropertiesAsync](#)). You should commit changes when the item loses focus because this triggers a selection reset.

Note that two-way binding using this technique works only with indexed locations, such as Music. You can determine whether a location is indexed by calling the [FolderInformation.GetIndexedStateAsync](#) method.

Note also that a virtualized vector can return **null** for some items before it populates their value. For example, you should check for **null** before you use the **SelectedItem** value of a list control bound to a virtualized vector, or use **SelectedIndex** instead.

Binding to data grouped by a key

If you take a flat collection of items (books, for example, represented by a **BookSku** class) and you group the items by using a common property as a key (the **BookSku.AuthorName** property, for example) then the result is called grouped data. When you group data, it is no longer a flat collection. Grouped data is a collection of group objects, where each group object has

- a key, and
- a collection of items whose property matches that key.

To take the books example again, the result of grouping the books by author name results in a collection of author name groups where each group has

- a key, which is an author name, and
- a collection of the **BookSkus** whose **AuthorName** property matches the group's key.

In general, to display a collection, you bind the **ItemsSource** of an items control (such as **ListView** or **GridView**) directly to a property that returns a collection. If that's a flat collection of items then you don't need to do anything special. But if it's a collection of group objects (as it is when binding to grouped data) then you need the services of an intermediary object called a **CollectionViewSource** which sits between the items control and the binding source. You bind the **CollectionViewSource** to the property that returns grouped data, and you bind the items control to the **CollectionViewSource**. An extra value-add of a **CollectionViewSource** is that it keeps track of the current item, so you can keep more than one items control in sync by binding them all to the same **CollectionViewSource**. You can also access the current item programmatically through the **ICollectionView.CurrentItem** property of the object returned by the **CollectionViewSource.View** property.

To activate the grouping facility of a **CollectionViewSource**, set **IsSourceGrouped** to **true**. Whether you also need to set the **ItemsPath** property depends on exactly how you author your group objects. There are two ways to author a group object: the "is-a-group" pattern, and the "has-a-group" pattern. In the "is-a-group" pattern, the group object derives from a collection type (for example, **List<T>**), so the group object actually is itself the group of items. With this pattern you do not need to set **ItemsPath**. In the "has-a-group" pattern, the group object has one or more properties of a collection type (such as **List<T>**), so the group "has a" group of items in the form of a property (or several groups of items in the form of several properties). With this pattern you need to set **ItemsPath** to the name of the property that contains the group of items.

The example below illustrates the "has-a-group" pattern. The page class has a property named **ViewModel**, which returns an instance of our view model. The **CollectionViewSource** binds to the **Authors** property of the view model (**Authors** is the collection of group objects) and also specifies that it's the **Author.BookSkus** property that contains the grouped items. Finally, the **GridView** is bound to the **CollectionViewSource**, and has its group style defined so that it can render the items in groups.

XAML

```
<Page.Resources>
    <CollectionViewSource
```

```

    x:Name="AuthorHasACollectionOfBookSku"
    Source="{x:Bind ViewModel.Authors}"
    IsSourceGrouped="true"
    ItemsPath="BookSkus"/>
</Page.Resources>
...
<GridView
ItemsSource="{x:Bind AuthorHasACollectionOfBookSku}" ...>
    <GridView.GroupStyle>
        <GroupStyle
            HeaderTemplate="{StaticResource AuthorGroupHeaderTemplateWide}" ... />
    </GridView.GroupStyle>
</GridView>

```

You can implement the "is-a-group" pattern in one of two ways. One way is to author your own group class. Derive the class from `List<T>` (where *T* is the type of the items). For example, `public class Author : List<BookSku>`. The second way is to use a [LINQ](#) expression to dynamically create group objects (and a group class) from like property values of the `BookSku` items. This approach—maintaining only a flat list of items and grouping them together on the fly—is typical of an app that accesses data from a cloud service. You get the flexibility to group books by author or by genre (for example) without needing special group classes such as `Author` and `Genre`.

The example below illustrates the "is-a-group" pattern using [LINQ](#). This time we group books by genre, displayed with the genre name in the group headers. This is indicated by the "Key" property path in reference to the group [Key](#) value.

```

C#

using System.Linq;
...
private IEnumerable<IGrouping<string, BookSku>> genres;

public IEnumerable<IGrouping<string, BookSku>> Genres
{
    get
    {
        if (this.genres == null)
        {
            this.genres = from book in this.bookSkus
                           group book by book.genre into grp
                           orderby grp.Key
                           select grp;
        }
        return this.genres;
    }
}

```

Remember that when using `{x:Bind}` with data templates we need to indicate the type being bound to by setting an `x:DataType` value. If the type is generic then we can't express that in markup so we need to use `{Binding}` instead in the group style header template.

```

XAML

<Grid.Resources>
    <CollectionViewSource x:Name="GenreIsACollectionOfBookSku"
        Source="{x:Bind Genres}"
        IsSourceGrouped="true"/>
</Grid.Resources>

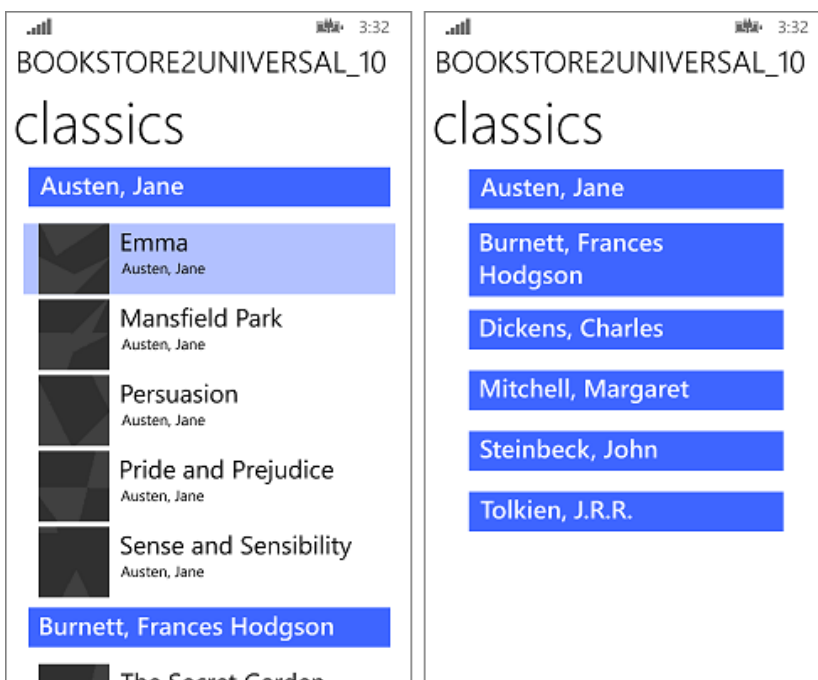
```

```

<GridView ItemsSource="{x:Bind GenreIsACollectionOfBookSku}">
  <GridView.ItemTemplate x:DataType="local:BookTemplate">
    <DataTemplate>
      <TextBlock Text="{x:Bind Title}"/>
    </DataTemplate>
  </GridView.ItemTemplate>
  <GridView.GroupStyle>
    <GroupStyle>
      <GroupStyle.HeaderTemplate>
        <DataTemplate>
          <TextBlock Text="{Binding Key}"/>
        </DataTemplate>
      </GroupStyle.HeaderTemplate>
    </GroupStyle>
  </GridView.GroupStyle>
</GridView>

```

A **SemanticZoom** control is a great way for your users to view and navigate grouped data. The [Bookstore2](#) sample app illustrates how to use the **SemanticZoom**. In that app, you can view a list of books grouped by author (the zoomed-in view) or you can zoom out to see a jump list of authors (the zoomed-out view). The jump list affords much quicker navigation than scrolling through the list of books. The zoomed-in and zoomed-out views are actually **ListView** or **GridView** controls bound to the same **CollectionViewSource**.



When you bind to hierarchical data—such as subcategories within categories—you can choose to display the hierarchical levels in your UI with a series of items controls. A selection in one items control determines the contents of subsequent items controls. You can keep the lists synchronized by binding each list to its own **CollectionViewSource** and binding the **CollectionViewSource** instances together in a chain. This is called a master/details (or list/details) view. For more info, see [How to bind to hierarchical data and create a master/details view](#).

Diagnosing and debugging data binding problems

Your binding markup contains the names of properties (and, for C#, sometimes fields and methods). So when you rename a property, you'll also need to change any binding that references it. Forgetting to do

that leads to a typical example of a data binding bug, and your app either won't compile or won't run correctly.

The binding objects created by `{x:Bind}` and `{Binding}` are largely functionally equivalent. But `{x:Bind}` has type information for the binding source, and it generates source code at compile-time. With `{x:Bind}` you get the same kind of problem detection that you get with the rest of your code. That includes compile-time validation of your binding expressions, and debugging by setting breakpoints in the source code generated as the partial class for your page. These classes can be found in the files in your `obj` folder, with names like (for C#) `<view name>.g.cs`). If you have a problem with a binding then turn on **Break On Unhandled Exceptions** in the Microsoft Visual Studio debugger. The debugger will break execution at that point, and you can then debug what has gone wrong. The code generated by `{x:Bind}` follows the same pattern for each part of the graph of binding source nodes, and you can use the info in the **Call Stack** window to help determine the sequence of calls that led up to the problem.

`{Binding}` does not have type information for the binding source. But when you run your app with the debugger attached, any binding errors appear in the **Output** window in Visual Studio.

Creating bindings in code

Note This section only applies to `{Binding}`, because you can't create `{x:Bind}` bindings in code. However, some of the same benefits of `{x:Bind}` can be achieved with `DependencyObject.RegisterPropertyChangedCallback`, which enables you to register for change notifications on any dependency property.

You can also connect UI elements to data using procedural code instead of XAML. To do this, create a new `Binding` object, set the appropriate properties, then call `FrameworkElement.SetBinding` or `BindingOperations.SetBinding`. Creating bindings programmatically is useful when you want to choose the binding property values at run-time or share a single binding among multiple controls. Note, however, that you cannot change the binding property values after you call `SetBinding`.

The following example shows how to implement a binding in code.

XAML

```
<TextBox x:Name="MyTextBox" Text="Text"/>
```

C#

```
// Create an instance of the MyColors class
// that implements INotifyPropertyChanged.
MyColors textcolor = new MyColors();

// Brush1 is set to be a SolidColorBrush with the value Red.
textcolor.Brush1 = new SolidColorBrush(Colors.Red);

// Set the DataContext of the TextBox MyTextBox.
MyTextBox.DataContext = textcolor;

// Create the binding and associate it with the text box.
Binding binding = new Binding() { Path = new PropertyPath("Brush1") };
MyTextBox.SetBinding(TextBox.ForegroundProperty, binding);
```

vbnet


```

' Create an instance of the MyColors class
' that implements INotifyPropertyChanged.
Dim textcolor As New MyColors()

' Brush1 is set to be a SolidColorBrush with the value Red.
textcolor.Brush1 = New SolidColorBrush(Colors.Red)

' Set the DataContext of the TextBox MyTextBox.
MyTextBox.DataContext = textcolor

' Create the binding and associate it with the text box.
Dim binding As New Binding() With {.Path = New PropertyPath("Brush1")}
MyTextBox.SetBinding(TextBox.ForegroundProperty, binding)

```

{x:Bind} and {Binding} feature comparison

Feature	{x:Bind} vs. {Binding}	Notes
Path is the default property	{x:Bind a.b.c} - {Binding a.b.c}	
Path property	{x:Bind Path=a.b.c} - {Binding Path=a.b.c}	In x:Bind, Path is rooted at the Page by default, not the DataContext.
Indexer	{x:Bind Groups[2].Title} - {Binding Groups[2].Title}	Binds to the specified item in the collection. Only integer-based indexes are supported.
Attached properties	{x:Bind Button22.(Grid.Row)} - {Binding Button22.(Grid.Row)}	Attached properties are specified using parentheses. If the property is not declared in a XAML namespace, then prefix it with an xml namespace, which should be mapped to a code namespace at the head of the document.
Casting	{x:Bind groups[0]. (data:SampleDataGroup.Title)} - Not needed for {Binding}.	Casts are specified using parentheses. If the property is not declared in a XAML namespace, then prefix it with an xml namespace, which should be mapped to a code namespace at the head of the document.
Converter	{x:Bind IsShown, Converter= {StaticResource BoolToVisibility}} - {Binding IsShown, Converter= {StaticResource BoolToVisibility}}	Converters must be declared at the root of the Page/ResourceDictionary, or in App.xaml.
ConverterParameter, ConverterLanguage	{x:Bind IsShown, Converter= {StaticResource BoolToVisibility}, ConverterParameter=One, ConverterLanguage=fr-fr} - {Binding IsShown, Converter= {StaticResource BoolToVisibility}, ConverterParameter=One, ConverterLanguage=fr-fr}	Converters must be declared at the root of the Page/ResourceDictionary, or in App.xaml.

Feature	{x:Bind} vs. {Binding}	Notes
TargetNullValue	<pre>{x:Bind Name, TargetNullValue=0}</pre> - <pre>{Binding Name, TargetNullValue=0}</pre>	Used when the leaf of the binding expression is null. Use single quotes for a string value.
FallbackValue	<pre>{x:Bind Name,</pre> <pre>FallbackValue='empty'}</pre> - <pre>{Binding Name,</pre> <pre>FallbackValue='empty'}</pre>	Used when any part of the path for the binding (except for the leaf) is null.
ElementName	<pre>{x:Bind slider1.Value}</pre> - <pre>{Binding Value,</pre> <pre>ElementName=slider1}</pre>	With {x:Bind} you're binding to a field; Path is rooted at the Page by default, so any named element can be accessed via its field.
RelativeSource: Self	<pre><Rectangle x:Name="rect1"</pre> <pre>Width="200" Height="{x:Bind</pre> <pre>rect1.Width}" ... /></pre> - <pre><Rectangle Width="200" Height="</pre> <pre>{Binding Width, RelativeSource=</pre> <pre>{RelativeSource Self}}" ... /></pre>	With {x:Bind}, name the element and use its name in Path.
RelativeSource: TemplatedParent	Not needed for {x:Bind} - <pre>{Binding <path>, RelativeSource=</pre> <pre>{RelativeSource TemplatedParent}}</pre>	With {x:Bind} TargetType on ControlTemplate indicates binding to template parent. For {Binding} Regular template binding can be used in control templates for most uses. But use TemplatedParent where you need to use a converter, or a two-way binding.<
Source	Not needed for {x:Bind} - <pre><ListView ItemsSource="{Binding</pre> <pre>Orders, Source={StaticResource</pre> <pre>MyData}}"/></pre>	For {x:Bind} you can directly use the named element, use a property or a static path.
Mode	<pre>{x:Bind Name, Mode=OneWay}</pre> - <pre>{Binding Name, Mode=TwoWay}</pre>	Mode can be OneTime, OneWay, or TwoWay. {x:Bind} defaults to OneTime; {Binding} defaults to OneWay.
UpdateSourceTrigger	<pre>{x:Bind Name, Mode=TwoWay,</pre> <pre>UpdateSourceTrigger=PropertyChanged}</pre> - <pre>{Binding</pre> <pre>UpdateSourceTrigger=PropertyChanged}</pre>	UpdateSourceTrigger can be Default, LostFocus, or PropertyChanged. {x:Bind} does not support UpdateSourceTrigger=Explicit. {x:Bind} uses PropertyChanged behavior for all cases except TextBox.Text, where it uses LostFocus behavior.

Functions in x:Bind

Article • 10/20/2022

ⓘ Note

For general info about using data binding in your app with `{x:Bind}` (and for an all-up comparison between `{x:Bind}` and `{Binding}`), see [Data binding in depth](#) and [{x:Bind} Markup Extension](#).

Starting in Windows 10, version 1607, `{x:Bind}` supports using a function as the leaf step of the binding path. This enables:

- A simpler way to achieve value conversion
- A way for bindings to depend on more than one parameter

ⓘ Note

To use functions with `{x:Bind}`, your app's minimum target SDK version must be 14393 or later. You can't use functions when your app targets earlier versions of Windows 10. For more info about target versions, see [Version adaptive code](#).

In the following example, the background and foreground of the item are bound to functions to do conversion based on the color parameter

XAML

```
<DataTemplate x:DataType="local:ColorEntry">
    <Grid Background="{x:Bind local:ColorEntry.Brushify(Color),
Mode=OneWay}" Width="240">
        <TextBlock Text="{x:Bind ColorName}" Foreground="{x:Bind
TextColor(Color)}" Margin="10,5" />
    </Grid>
</DataTemplate>
```

C#

```
class ColorEntry
{
    public string ColorName { get; set; }
    public Color Color { get; set; }

    public static SolidColorBrush Brushify(Color c)
    {
```

```

        return new SolidColorBrush(c);
    }

    public SolidColorBrush TextColor(Color c)
    {
        return new SolidColorBrush(((c.R * 0.299 + c.G * 0.587 + c.B *
0.114) > 150) ? Colors.Black : Colors.White);
    }
}

```

XAML attribute usage

XAML

```

<object property="{x:Bind pathToFunction.FunctionName(functionParameter1,
functionParameter2, ...), bindingProperties}" ... />

```

Path to the function

The [path to the function](#) is specified like other property paths and can include [dots](#) (.), [indexers](#) or [casts](#) to locate the function.

Static functions can be specified using XMLNamespace:ClassName.MethodName syntax. For example, use the below syntax for binding to static functions in code-behind.

XAML

```

<Page
    xmlns:local="using:MyNamespace">
    ...
    <StackPanel>
        <TextBlock x:Name="BigTextBlock" FontSize="20" Text="Big text" />
        <TextBlock FontSize="{x:Bind
local:MyHelpers.Half(BigTextBlock.FontSize)}"
            Text="Small text" />
    </StackPanel>
</Page>

```

C#

```

namespace MyNamespace
{
    static public class MyHelpers
    {
        public static double Half(double value) => value / 2.0;
    }
}

```

```
}  
}
```

You can also use system functions directly in markup to accomplish simple scenarios like date formatting, text formatting, text concatenations, etc., For example:

XAML

```
<Page  
    xmlns:sys="using:System"  
    xmlns:local="using:MyNamespace">  
    ...  
    <CalendarDatePicker Date="{x:Bind sys:DateTime.Parse(TextBlock1.Text)}"  
/>  
    <TextBlock Text="{x:Bind sys:String.Format('{0} is now available in  
{1}', local:MyPage.personName, local:MyPage.location)}" />  
</Page>
```

If the mode is OneWay/TwoWay, then the function path will have change detection performed on it, and the binding will be re-evaluated if there are changes to those objects.

The function being bound to needs to:

- Be accessible to the code and metadata – so internal / private work in C#, but C++/CX will need methods to be public WinRT methods
- Overloading is based on the number of arguments, not type, and it will try to match to the first overload with that many arguments
- The argument types need to match the data being passed in – we don't do narrowing conversions
- The return type of the function needs to match the type of the property that is using the binding

The binding engine reacts to property change notifications fired with the function name and re-evaluate bindings as necessary. For example:

XAML

```
<DataTemplate x:DataType="local:Person">  
    <StackPanel>  
        <TextBlock Text="{x:Bind FullName}" />  
        <Image Source="{x:Bind IconToBitmap(Icon, CancellationToken),  
Mode=OneWay}" />  
    </StackPanel>  
</DataTemplate>
```

C#

```
public class Person : INotifyPropertyChanged
{
    //Implementation for an Icon property and a CancellationToken property
    with PropertyChanged notifications
    ...

    //IconToBitmap function is essentially a multi binding converter between
    several options.
    public Uri IconToBitmap (Uri icon, Uri cancellationToken)
    {
        Uri foo = new Uri(...);
        if (isCancelled)
        {
            foo = cancellationToken;
        }
        else
        {
            if (this.fullName.Contains("Sr"))
            {
                //pass a different Uri back
                foo = new Uri(...);
            }
            else
            {
                foo = icon;
            }
        }
        return foo;
    }

    //Ensure FullName property handles change notification on itself as well
    as IconToBitmap since the function uses it
    public string FullName
    {
        get { return this.fullName; }
        set
        {
            this.fullName = value;
            this.OnPropertyChanged ();
            this.OnPropertyChanged ("IconToBitmap");
            //this ensures Image.Source binding re-evaluates when FullName
            changes in addition to Icon and CancellationToken
        }
    }
}
```

Tip

You can use functions in x:Bind to achieve the same scenarios as what was supported through Converters and MultiBinding in WPF.

Function arguments

Multiple function arguments can be specified, separated by comma's (,)

- Binding Path – Same syntax as if you were binding directly to that object.
 - If the mode is OneWay/TwoWay then change detection will be performed and the binding re-evaluated upon object changes
- Constant string enclosed in quotes – quotes are needed to designate it as a string. Hat (^) can be used to escape quotes in strings
- Constant Number - for example -123.456
- Boolean – specified as "x:True" or "x:False"

Two way function bindings

In a two-way binding scenario, a second function must be specified for the reverse direction of the binding. This is done using the **BindBack** binding property. In the below example, the function should take one argument which is the value that needs to be pushed back to the model.

XAML

```
<TextBlock Text="{x:Bind a.MyFunc(b), BindBack=a.MyFunc2, Mode=TwoWay}" />
```

See also

- [{x:Bind} Markup Extension](#)

Tutorial: Create data bindings

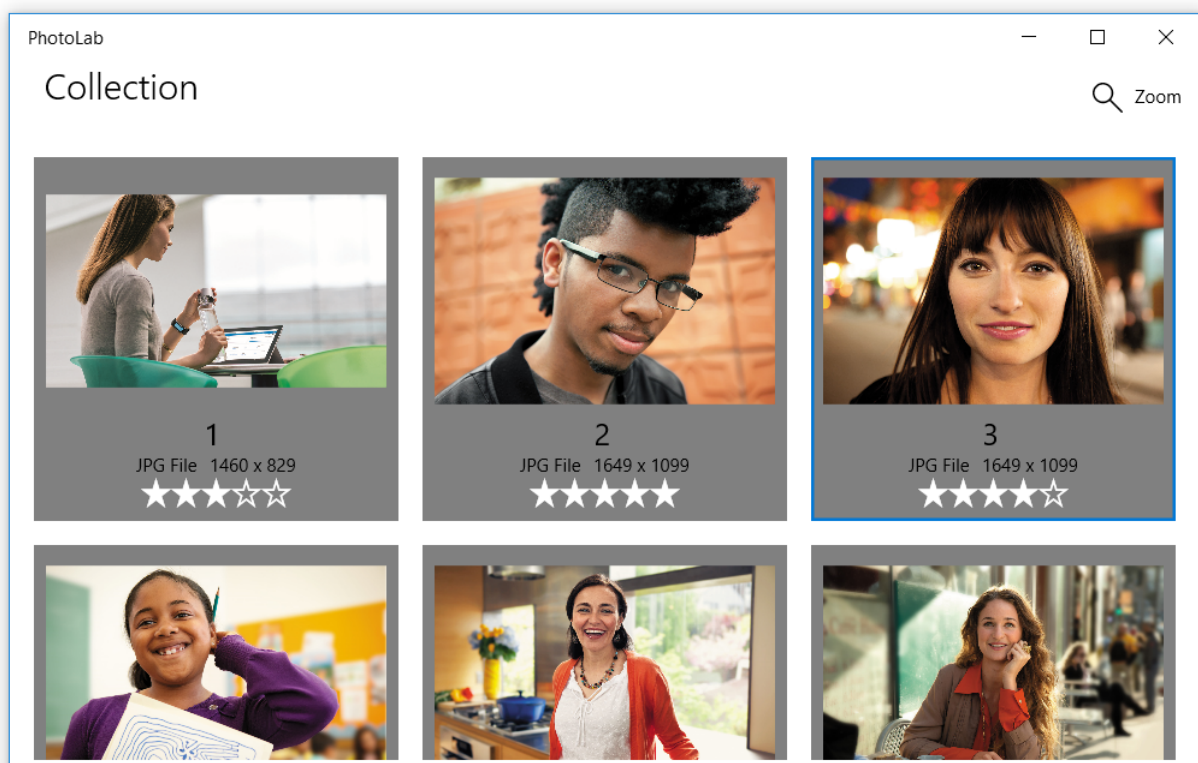
Article • 04/14/2023

Suppose you've designed and implemented a nice looking UI filled with placeholder images, "lorem ipsum" boilerplate text, and controls that don't do anything yet. Next, you'll want to connect it to real data and transform it from a design prototype into a living app.

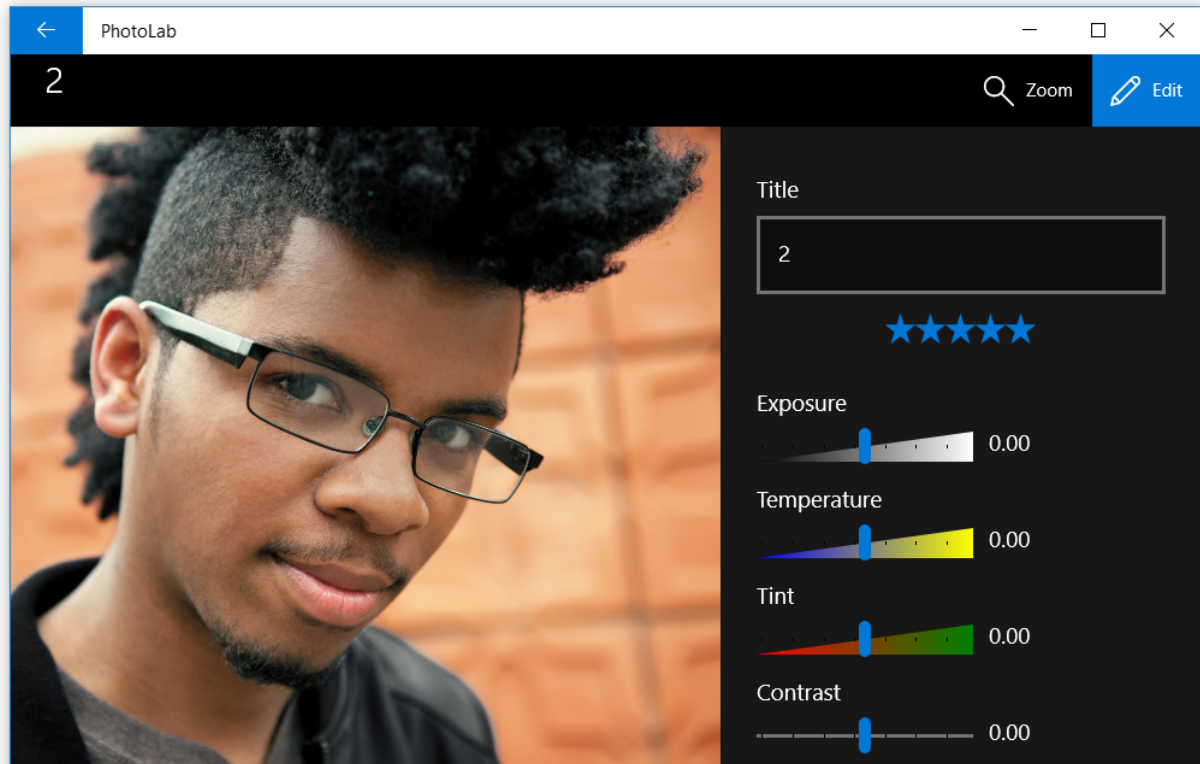
In this tutorial, you'll learn how to replace your boilerplate with data bindings and create other direct links between your UI and your data. You'll also learn how to format or convert your data for display, and keep your UI and data in sync. When you complete this tutorial, you'll be able to improve the simplicity and organization of the XAML and C# code, making it easier to maintain and extend.

You'll start with a simplified version of the PhotoLab sample. This starter version includes the complete data layer plus the basic XAML page layouts, and leaves out many features to make the code easier to browse around in. This tutorial doesn't build up to the complete app, so be sure to check out the final version to see features such as custom animations and adaptive layouts. You can find the final version in the root folder of the [Windows-appsample-photo-lab](#) repo.

The PhotoLab sample app has two pages. The *main page* displays a photo gallery view, along with some information about each image file.



The *details* page displays a single photo after it has been selected. A flyout editing menu allows the photo to be altered, renamed, and saved.



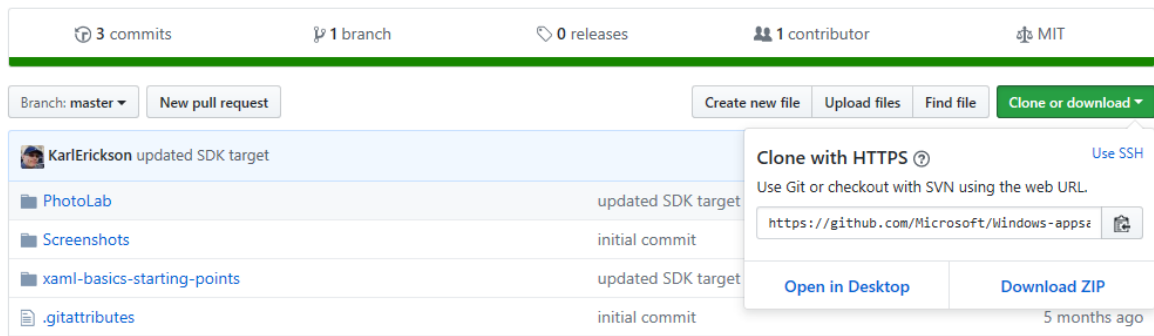
Prerequisites

- Visual Studio 2019 or later: [Download Visual Studio](#) (The Community edition is free.)
- Windows SDK (10.0.17763.0 or later): [Download the latest Windows SDK \(free\)](#)
- Windows 10, Version 1809 or later

Part 0: Get the starter code from GitHub

For this tutorial, you'll start with a simplified version of the PhotoLab sample.

1. Go to the GitHub page for the sample: <https://github.com/Microsoft/Windows-appsample-photo-lab>.
2. Next, you'll need to clone or download the sample. Select the **Clone or download** button. A sub-menu appears.



If you're not familiar with GitHub:

- Select **Download ZIP** and save the file locally. This downloads a .zip file that contains all the project files you need.
- Extract the file. Use File Explorer to browse to the .zip file that you just downloaded, right-click it, and select **Extract All...**
- Browse to your local copy of the sample, and go to the `Windows-appsample-photo-lab-master\xaml-basics-starting-points\data-binding` directory.

If you are familiar with GitHub:

- Clone the main branch of the repo locally.
 - Browse to the `Windows-appsample-photo-lab\xaml-basics-starting-points\data-binding` directory.
3. Double-click `PhotoLab.sln` to open the solution in Visual Studio.

Part 1: Replace the placeholders

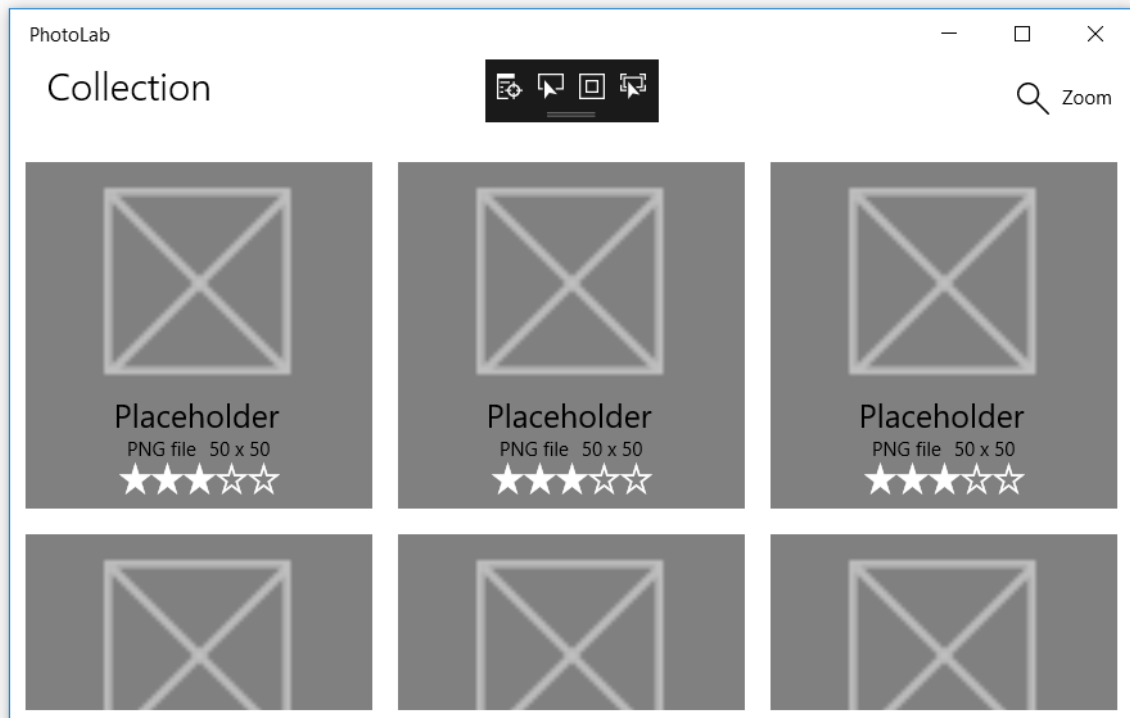
Here, you create one-time bindings in data-template XAML to display real images and image metadata instead of placeholder content.

One-time bindings are for read-only, unchanging data, which means they are high performance and easy to create, letting you display large data sets in `GridView` and `ListView` controls.

Replace the placeholders with one-time bindings

- Open the `xaml-basics-starting-points\data-binding` folder and launch the `PhotoLab.sln` file in Visual Studio.

2. Make sure your **Solution Platform** is set to x86 or x64, not Arm, and then run the app. This shows the state of the app with UI placeholders, before bindings have been added.



3. Open MainPage.xaml and search for a `DataTemplate` named **ImageGridView_DefaultItemTemplate**. You'll update this template to use data bindings.

Before:

XAML

```
<DataTemplate x:Key="ImageGridView_DefaultItemTemplate">
```

The `x:Key` value is used by the `ImageGridView` to select this template for displaying data objects.

4. Add an `x:DataType` value to the template.

After:

XAML

```
<DataTemplate x:Key="ImageGridView_DefaultItemTemplate"
              x:DataType="local:ImageFileInfo">
```

`x:DataType` indicates which type this is a template for. In this case, it's a template for the `ImageFileInfo` class (where `local:` indicates the local namespace, as defined in an xmlns declaration near the top of the file).

`x:DataType` is required when using `x:Bind` expressions in a data template, as described next.

5. In the `DataTemplate`, find the `Image` element named `ItemImage` and replace its `Source` value as shown.

Before:

XAML

```
<Image x:Name="ItemImage"
       Source="/Assets/StoreLogo.png"
       Stretch="Uniform" />
```

After:

XAML

```
<Image x:Name="ItemImage"
       Source="{x:Bind ImageSource}"
       Stretch="Uniform" />
```

`x:Name` identifies a XAML element so you can refer to it elsewhere in the XAML and in the code-behind.

`x:Bind` expressions supply a value to a UI property by getting the value from a **data-object** property. In templates, the property indicated is a property of whatever the `x:DataType` has been set to. So in this case, the data source is the `ImageFileInfo.ImageSource` property.

ⓘ Note

The `x:Bind` value also lets the editor know about the data type, so you can use IntelliSense instead of typing in the property name in an `x:Bind` expression. Try it on the code you just pasted in: place the cursor just after `x:Bind` and press the Spacebar to see a list of properties you can bind to.

6. Replace the values of the other UI controls in the same way. (Try doing this with IntelliSense instead of copy/pasting!)

Before:

XAML

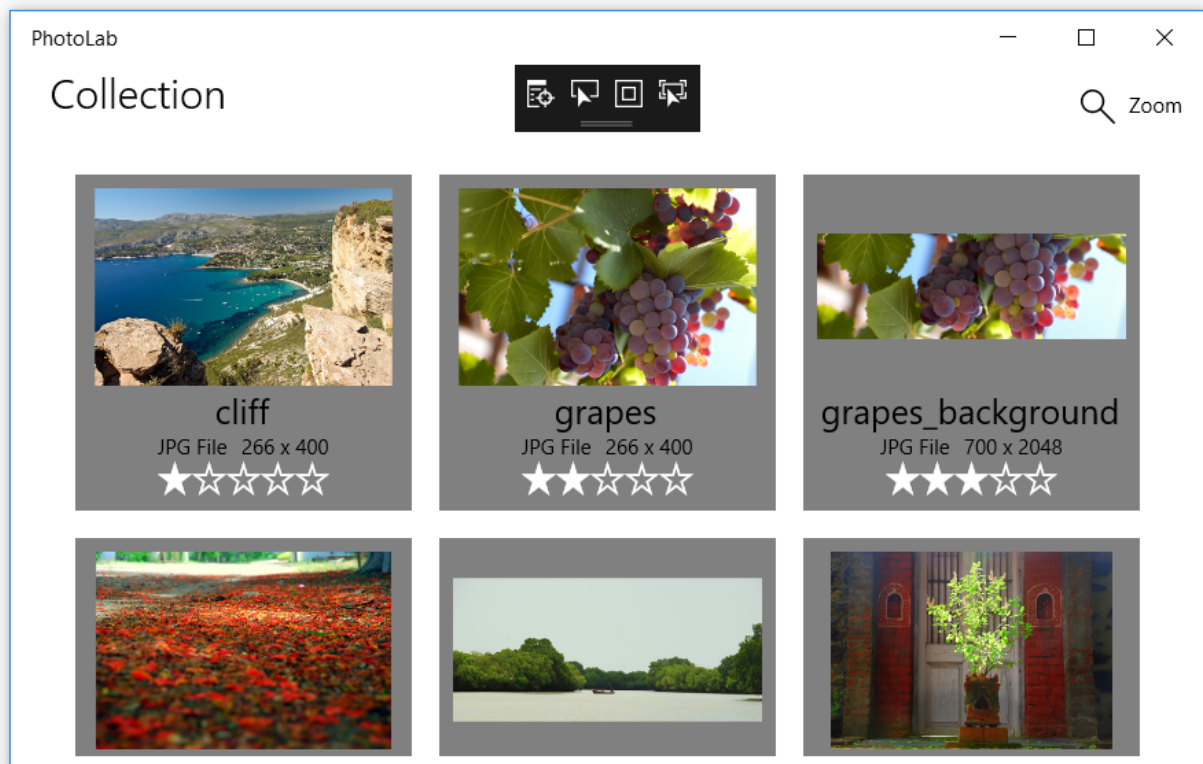
```
<TextBlock Text="Placeholder" ... />
<StackPanel ... >
    <TextBlock Text="PNG file" ... />
    <TextBlock Text="50 x 50" ... />
</StackPanel>
<muxc:RatingControl Value="3" ... />
```

After:

XAML

```
<TextBlock Text="{x:Bind ImageTitle}" ... />
<StackPanel ... >
    <TextBlock Text="{x:Bind ImageFileType}" ... />
    <TextBlock Text="{x:Bind ImageDimensions}" ... />
</StackPanel>
<muxc:RatingControl Value="{x:Bind ImageRating}" ... />
```

Run the app to see how it looks so far. No more placeholders! We're off to a good start.



ⓘ Note

If you want to experiment further, try adding a new TextBlock to the data template, and use the `x:Bind` IntelliSense trick to find a property to display.

Part 2: Use binding to connect the gallery UI to the images

Here, you'll create one-time bindings in page XAML to connect the gallery view to the image collection, replacing the existing procedural code that does this in code-behind. You'll also create a **Delete** button to see how the gallery view changes when images are removed from the collection. At the same time, you'll learn how to bind events to event handlers for more flexibility than that provided by traditional event handlers.

All the bindings covered so far are inside data templates and refer to properties of the class indicated by the `x:DataType` value. What about the rest of the XAML in your page?

`x:Bind` expressions outside of data templates are always bound to the page itself. This means you can reference anything you put in code-behind or declare in XAML, including custom properties and properties of other UI controls on the page (as long as they have an `x:Name` value).

In the PhotoLab sample, one use for a binding like this is to connect the main `GridView` control directly to the collection of images, instead of doing it in code-behind. Later, you'll see other examples.

Bind the main GridView control to the Images collection

1. In `MainPage.xaml.cs`, find the `GetItemsAsync` method and remove the code that sets `ItemsSource`.

Before:

```
C#  
  
ImageGridView.ItemsSource = Images;
```

After:

```
C#  
  
// Replaced with XAML binding:  
// ImageGridView.ItemsSource = Images;
```

2. In MainPage.xaml, find the `GridView` named `ImageGridView` and add an `ItemsSource` attribute. For the value, use an `x:Bind` expression that refers to the `Images` property implemented in code-behind.

Before:

XAML

```
<GridView x:Name="ImageGridView"
```

After:

XAML

```
<GridView x:Name="ImageGridView"  
    ItemsSource="{x:Bind Images}"
```

The `Images` property is of type `ObservableCollection<ImageFileInfo>`, so the individual items displayed in the `GridView` are of type `ImageFileInfo`. This matches the `x:DataType` value described in Part 1.

All the bindings we've looked at so far are one-time, read-only bindings, which is the default behavior for plain `x:Bind` expressions. The data is loaded only at initialization, which makes for high-performance bindings - perfect for supporting multiple, complex views of large data sets.

Even the `ItemsSource` binding you just added is a one-time, read-only binding to an unchanging property value, but there's an important distinction to make here. The unchanging value of the `Images` property is a single, specific instance of a collection, initialized once as shown here.

C#

```
private ObservableCollection<ImageFileInfo> Images { get; }  
    = new ObservableCollection<ImageFileInfo>();
```

The `Images` property value never changes, but because the property is of type `ObservableCollection<T>`, the *contents* of the collection can change, and the binding will automatically notice the changes and update the UI.

To test this, we're going to temporarily add a button that deletes the currently-selected image. This button isn't in the final version because selecting an image will take you to a detail page. However, the behavior of `ObservableCollection<T>` is still important in the

final PhotoLab sample because the XAML is initialized in the page constructor (through the `InitializeComponent` method call), but the `Images` collection is populated later in the `GetItemsAsync` method.

Add a delete button

1. In `MainPage.xaml`, find the `CommandBar` named `MainCommandBar` and add a new button before the zoom button. (The zoom controls don't work yet. You'll hook those up in the next part of the tutorial.)

XAML

```
<AppBarButton Icon="Delete"
              Label="Delete selected image"
              Click="{x:Bind DeleteSelectedImage}" />
```

If you're already familiar with XAML, this `Click` value might look unusual. In previous versions of XAML, you had to set this to a method with a specific event-handler signature, typically including parameters for the event sender and an event-specific arguments object. You can still use this technique when you need the event arguments, but with `x:Bind`, you can connect to other methods, too. For example, if you don't need the event data, you can connect to methods that have no parameters, like we do here.

2. In `MainPage.xaml.cs`, add the `DeleteSelectedImage` method.

C#

```
private void DeleteSelectedImage() =>
    Images.Remove(ImageGridView.SelectedItem as ImageFileInfo);
```

This method simply deletes the selected image from the `Images` collection.

Now run the app and use the button to delete a few images. As you can see, the UI is updated automatically, thanks to data binding and the `ObservableCollection<T>` type.

ⓘ Note

This code only deletes the `ImageFileInfo` instance from the `Images` collection in the running app. It does not delete the image file from the computer.

Part 3: Set up the zoom slider

In this part, you'll create one-way bindings from a control in the data template to the zoom slider, which is outside the template. You'll also learn that you can use data binding with many control properties, not just the most obvious ones like

`TextBlock.Text` and `Image.Source`.

Bind the image data template to the zoom slider

- Find the `DataTemplate` named `ImageGridView_DefaultItemTemplate` and replace the `**Height**` and `Width` values of the `Grid` control at the top of the template.

Before

XAML

```
<DataTemplate x:Key="ImageGridView_DefaultItemTemplate"
              x:DataType="local:ImageFileInfo">
  <Grid Height="200"
        Width="200"
        Margin="{StaticResource LargeItemMargin}">
```

After

XAML

```
<DataTemplate x:Key="ImageGridView_DefaultItemTemplate"
              x:DataType="local:ImageFileInfo">
  <Grid Height="{Binding Value, ElementName=ZoomSlider}"
        Width="{Binding Value, ElementName=ZoomSlider}"
        Margin="{StaticResource LargeItemMargin}">
```

Did you notice that these are `Binding` expressions, and not `x:Bind` expressions? This is the old way of doing data bindings, and it's mostly obsolete. `x:Bind` does nearly everything that `Binding` does, and more. However, when you use `x:Bind` in a data template, it binds to the type declared in the `x:DataType` value. So how do you bind something in the template to something in the page XAML or in code-behind? You must use an old-style `Binding` expression.

`Binding` expressions don't recognize the `x:DataType` value, but these `Binding` expressions have `ElementName` values that work almost the same way. These tell the binding engine that **Binding Value** is a binding to the `Value` property of the specified element on the page (that is, the element with that `x:Name` value). If you want to bind to

a property in code-behind, it would look something like `{Binding MyCodeBehindProperty, ElementName=page}` where `page` refers to the `x:Name` value set in the `Page` element in XAML.

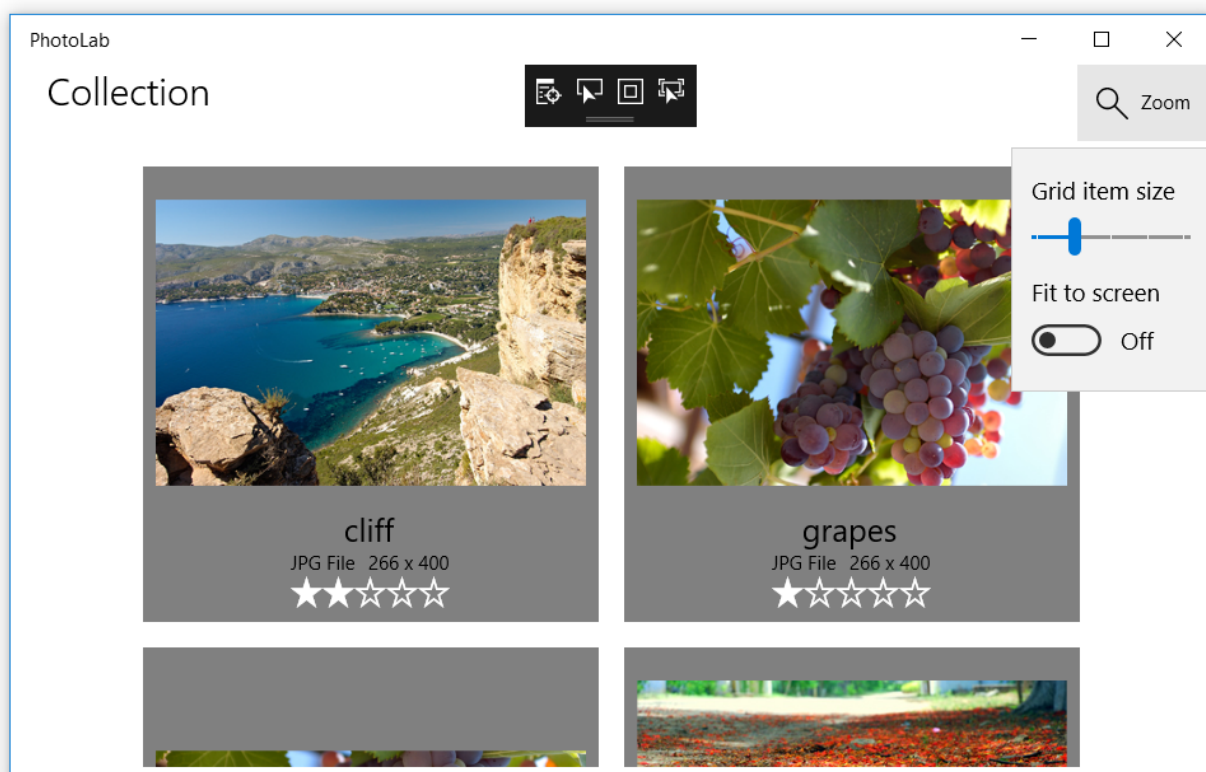
❗ Note

By default, `Binding` expressions are one-way, meaning that they will automatically update the UI when the bound property value changes.

In contrast, the default for `x:Bind` is one-time, meaning that any changes to the bound property are ignored. This is the default because it's the most high-performance option, and most bindings are to static, read-only data.

The lesson here is that if you use `x:Bind` with properties that can change their values, be sure to add `Mode=OneWay` or `Mode=TwoWay`. You'll see examples of this in the next section.

Run the app and use the slider to change the image-template dimensions. As you can see, the effect is pretty powerful without needing much code.



❗ Note

For a challenge, try binding other UI properties to the zoom slider `value` property, or to other sliders that you add after the zoom slider. For example, you could bind

the `FontSize` property of the `TitleTextBlock` to a new slider with a default value of 24. Be sure to set reasonable minimum and maximum values.

Part 4: Improve the zoom experience

In this part, you'll add a custom `ItemSize` property to code-behind and create one-way bindings from the image template to the new property. The `ItemSize` value will be updated by the zoom slider and other factors such as the **Fit to screen** toggle and the window size, making for a more refined experience.

Unlike built-in control properties, your custom properties do not automatically update the UI, even with one-way and two-way bindings. They work fine with one-*time* bindings, but if you want your property changes to actually show up in your UI, you need to do some work.

Create the `ItemSize` property so that it updates the UI

1. In `MainPage.xaml.cs`, change the signature of the `MainPage` class so that it implements the `INotifyPropertyChanged` interface.

Before:

```
C#  
  
public sealed partial class MainPage : Page
```

After:

```
C#  
  
public sealed partial class MainPage : Page, INotifyPropertyChanged
```

This informs the binding system that `MainPage` has a `PropertyChanged` event (added next) that bindings can listen for in order to update the UI.

2. Add a `PropertyChanged` event to the `MainPage` class.

```
C#  
  
public event PropertyChangedEventHandler PropertyChanged;
```

This event provides the complete implementation required by the `INotifyPropertyChanged` interface. However, for it to have any effect, you must explicitly raise the event in your custom properties.

3. Add an `ItemSize` property and raise the `PropertyChanged` event in its setter.

```
C#  
  
public double ItemSize  
{  
    get => _itemSize;  
    set  
    {  
        if (_itemSize != value)  
        {  
            _itemSize = value;  
            PropertyChanged?.Invoke(this, new  
PropertyChangedEventArgs(nameof(ItemSize)));  
        }  
    }  
}  
private double _itemSize;
```

The `ItemSize` property exposes the value of a private `_itemSize` field. Using a backing field like this enables the property to check whether a new value is the same as the old value before it raises a potentially unnecessary `PropertyChanged` event.

The event itself is raised by the `Invoke` method. The question mark checks whether the `PropertyChanged` event is null - that is, whether any event handlers have been added yet. Every one-way or two-way binding adds an event handler behind the scenes, but if no one is listening, nothing more would happen here. If `PropertyChanged` isn't null, however, then `Invoke` is called with a reference to the event source (the page itself, represented by the `this` keyword) and an **event-args** object that indicates the name of the property. With this info, any one-way or two-way bindings to the `ItemSize` property will be informed of any changes so they can update the bound UI.

4. In `MainPage.xaml`, find the `DataTemplate` named `ImageGridView_DefaultItemTemplate` and replace the `Height` and `Width` values of the `Grid` control at the top of the template. (If you did the control-to-control binding in the previous part of this tutorial, the only changes are to replace `Value` with `ItemSize` and `ZoomSlider` with `page`. Be sure to do this for both `Height` and `Width`!)

Before

XAML

```
<DataTemplate x:Key="ImageGridView_DefaultItemTemplate"
              x:DataType="local:ImageFileInfo">
    <Grid Height="{Binding Value, ElementName=ZoomSlider}"
          Width="{Binding Value, ElementName=ZoomSlider}"
          Margin="{StaticResource LargeItemMargin}">
```

After

XAML

```
<DataTemplate x:Key="ImageGridView_DefaultItemTemplate"
              x:DataType="local:ImageFileInfo">
    <Grid Height="{Binding ItemSize, ElementName=page}"
          Width="{Binding ItemSize, ElementName=page}"
          Margin="{StaticResource LargeItemMargin}">
```

Now that the UI can respond to `ItemSize` changes, you need to actually make some changes. As mentioned previously, the `ItemSize` value is calculated from the current state of various UI controls, but the calculation must be performed whenever those controls change state. To do this, you'll use event binding so that certain UI changes will call a helper method that updates `ItemSize`.

Update the `ItemSize` property value

1. Add the `DetermineItemSize` method to `MainPage.xaml.cs`.

C#

```
private void DetermineItemSize()
{
    if (FitScreenToggle != null
        && FitScreenToggle.IsOn == true
        && ImageGridView != null
        && ZoomSlider != null)
    {
        // The 'margins' value represents the total of the margins
        around the
        // image in the grid item. 8 from the ItemTemplate root grid +
        8 from
        // the ItemContainerStyle * (Right + Left). If those values
        change,
        // this value needs to be updated to match.
        int margins = (int)this.Resources["LargeItemMarginValue"] * 4;
        double gridWidth = ImageGridView.ActualWidth -
```

```

        (int)this.Resources["DefaultWindowSidePaddingValue"];
        double ItemWidth = ZoomSlider.Value + margins;
        // We need at least 1 column.
        int columns = (int)Math.Max(gridWidth / ItemWidth, 1);

        // Adjust the available grid width to account for margins
        around each item.
        double adjustedGridWidth = gridWidth - (columns * margins);

        ItemSize = (adjustedGridWidth / columns);
    }
    else
    {
        ItemSize = ZoomSlider.Value;
    }
}

```

2. In MainPage.xaml, navigate to the top of the file and add a `SizeChanged` event binding to the `Page` element.

Before:

XAML

```
<Page x:Name="page"
```

After:

XAML

```
<Page x:Name="page"
      SizeChanged="{x:Bind DetermineItemSize}"

```

3. Find the `Slider` named `ZoomSlider` (in the `Page.Resources` section) and add a `ValueChanged` event binding.

Before:

XAML

```
<Slider x:Name="ZoomSlider"

```

After:

XAML

```
<Slider x:Name="ZoomSlider"  
        ValueChanged="{x:Bind DetermineItemSize}"
```

4. Find the `ToggleSwitch` named `FitScreenToggle` and add a `Toggled` event binding.

Before:

XAML

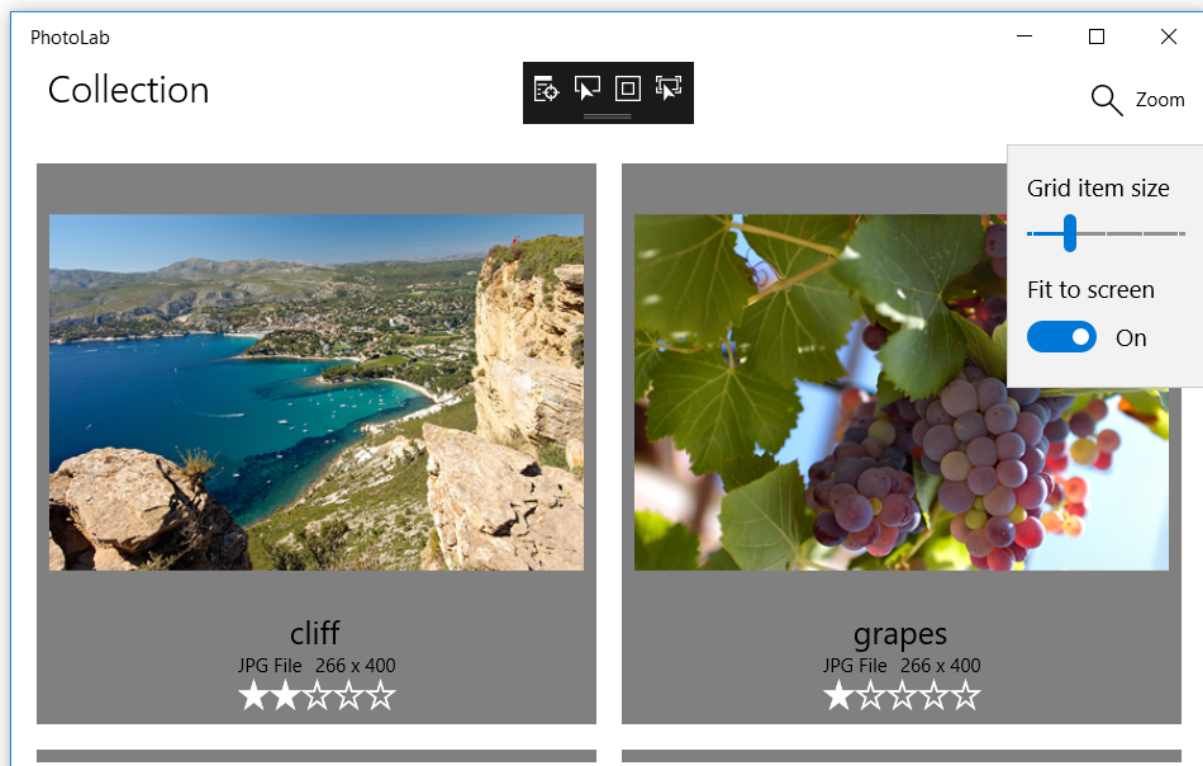
```
<ToggleSwitch x:Name="FitScreenToggle"
```

After:

XAML

```
<ToggleSwitch x:Name="FitScreenToggle"  
              Toggled="{x:Bind DetermineItemSize}"
```

Run the app and use the zoom slider and **Fit to screen** toggle to change the image-template dimensions. As you can see, the latest changes enable a more refined zoom/resize experience while keeping the code well organized.



📌 Note

For a challenge, try adding a `TextBlock` after the `ZoomSlider` and binding the `Text` property to the `ItemSize` property. Because it's not in a data template, you can use `x:Bind` instead of `Binding` like in the previous `ItemSize` bindings.

Part 5: Enable user edits

Here, you'll create two-way bindings to enable users to update values, including the image title, rating, and various visual effects.

To achieve this, you'll update the existing `DetailPage`, which provides a single-image viewer, zoom control, and editing UI.

First, however, you need to attach the `DetailPage` so that the app navigates to it when the user clicks an image in the gallery view.

Attach the DetailPage

1. In `MainPage.xaml`, find the `GridView` named `ImageGridView`. To make the items clickable, set `ItemClickEnabled` to `True` and add an `ItemClick` event handler.

Tip

If you type in the change below instead of copy/pasting, you'll see an IntelliSense pop-up that says "<New Event Handler>". If you press the Tab key, it will fill in the value with a default method handler name, and automatically stub out the method shown in the next step. You can then press F12 to navigate to the method in the code-behind.

Before:

XAML

```
<GridView x:Name="ImageGridView">
```

After:

XAML

```
<GridView x:Name="ImageGridView"
    IsItemClickEnabled="True"
    ItemClick="ImageGridView_ItemClick">
```


⚠ Note

We're using a conventional event handler here instead of an `x:Bind` expression. This is because we need to see the event data, as shown next.

2. In `MainPage.xaml.cs`, add the event handler (or fill it in, if you used the tip in the last step).

```
C#  
  
private void ImageGridView_ItemClick(object sender, ItemClickEventArgs e)  
{  
    this.Frame.Navigate(typeof(DetailPage), e.ClickedItem);  
}
```

This method simply navigates to the detail page, passing in the clicked item, which is an `ImageFileInfo` object used by `DetailPage.OnNavigatedTo` for initializing the page. You won't have to implement that method in this tutorial, but you can take a look to see what it does.

3. (Optional) Delete or comment out any controls you added in previous play-points that work with the currently selected image. Keeping them around won't hurt anything, but it's now a lot harder to select an image without navigating to the detail page.

Now that you've connected the two pages, run the app and take a look around. Everything works except the controls on the editing pane, which don't respond when you try to change the values.

As you can see, the title text box does display the title and lets you type in changes. You have to change focus to another control to commit the changes, but the title in the upper-left corner of the screen doesn't update yet.

All the controls are already bound using the plain `x:Bind` expressions we covered in Part 1. If you recall, this means they are all one-time bindings, which explains why changes to the values aren't registered. To fix this, all we have to do is turn them into two-way bindings.

Make the editing controls interactive

1. In `DetailPage.xaml`, find the `TextBlock` named `TitleTextBlock` and the `RatingControl` control after it, and update their `x:Bind` expressions to include `Mode=TwoWay`.

Before:

XAML

```
<TextBlock x:Name="TitleTextBlock"
    Text="{x:Bind item.ImageTitle}"
    ... >
<muxc:RatingControl Value="{x:Bind item.ImageRating}"
    ... >
```

After:

XAML

```
<TextBlock x:Name="TitleTextBlock"
    Text="{x:Bind item.ImageTitle, Mode=TwoWay}"
    ... >
<muxc:RatingControl Value="{x:Bind item.ImageRating, Mode=TwoWay}"
    ... >
```

2. Do the same thing for all the effect sliders that come after the rating control.

XAML

```
<Slider Header="Exposure" ... Value="{x:Bind item.Exposure,
Mode=TwoWay}" ...
<Slider Header="Temperature" ... Value="{x:Bind item.Temperature,
Mode=TwoWay}" ...
<Slider Header="Tint" ... Value="{x:Bind item.Tint,
Mode=TwoWay}" ...
<Slider Header="Contrast" ... Value="{x:Bind item.Contrast,
Mode=TwoWay}" ...
<Slider Header="Saturation" ... Value="{x:Bind item.Saturation,
Mode=TwoWay}" ...
<Slider Header="Blur" ... Value="{x:Bind item.Blur,
Mode=TwoWay}" ...
```

The two-way mode, as you might expect, means that the data moves in both directions whenever there are changes on either side.

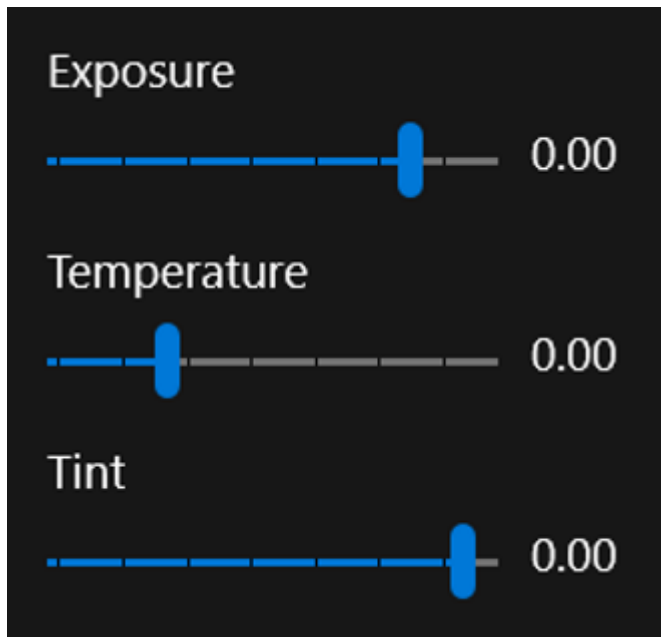
Like the one-way bindings covered earlier, these two-way bindings will now update the UI whenever the bound properties change, thanks to the `INotifyPropertyChanged` implementation in the `ImageFileInfo` class. With two-way binding, however, the values

will also move from the UI to the bound properties whenever the user interacts with the control. Nothing more is needed on the XAML side.

Run the app and try the editing controls. As you can see, when you make a change, it now affects the image values, and those changes persist when you navigate back to the main page.

Part 6: Format values through function binding

One last problem remains. When you move the effect sliders, the labels next to them still don't change.



The final part in this tutorial is to add bindings that format the slider values for display.

Bind the effect-slider labels and format the values for display

1. Find the `TextBlock` after the `Exposure` slider and replace the `Text` value with the binding expression shown here.

Before:

XAML

```
<Slider Header="Exposure" ... />
<TextBlock ... Text="0.00" />
```

After:

XAML

```
<Slider Header="Exposure" ... />
<TextBlock ... Text="{x:Bind item.Exposure.ToString('N', culture),
Mode=OneWay}" />
```

This is called a function binding because you are binding to the return value of a method. The method must be accessible through the page's code-behind or the `x:DataType` type if you are in a data template. In this case, the method is the familiar .NET `ToString` method, which is accessed through the item property of the page, and then through the `Exposure` property of the item. (This illustrates how you can bind to methods and properties that are deeply nested in a chain of connections.)

Function binding is an ideal way to format values for display because you can pass in other binding sources as method arguments, and the binding expression will listen for changes to those values as expected with the one-way mode. In this example, the `culture` argument is a reference to an unchanging field implemented in code-behind, but it could just as easily have been a property that raises `PropertyChanged` events. In that case, any changes to the property value would cause the `x:Bind` expression to call `ToString` with the new value and then update the UI with the result.

2. Do the same thing for the `TextBlocks` that label the other effect sliders.

XAML

```
<Slider Header="Temperature" ... />
<TextBlock ... Text="{x:Bind item.Temperature.ToString('N', culture),
Mode=OneWay}" />

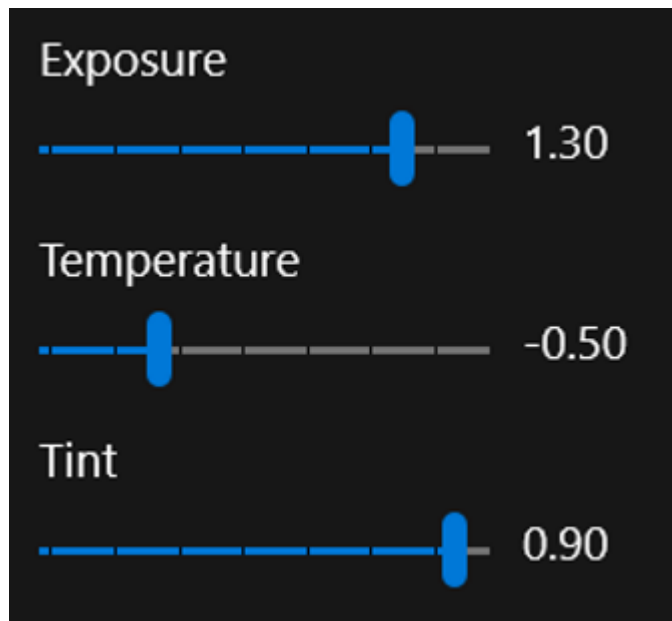
<Slider Header="Tint" ... />
<TextBlock ... Text="{x:Bind item.Tint.ToString('N', culture),
Mode=OneWay}" />

<Slider Header="Contrast" ... />
<TextBlock ... Text="{x:Bind item.Contrast.ToString('N', culture),
Mode=OneWay}" />

<Slider Header="Saturation" ... />
<TextBlock ... Text="{x:Bind item.Saturation.ToString('N', culture),
Mode=OneWay}" />

<Slider Header="Blur" ... />
<TextBlock ... Text="{x:Bind item.Blur.ToString('N', culture),
Mode=OneWay}" />
```

Now when you run the app, everything works, including the slider labels.



Conclusion

This tutorial has given you a taste of data binding and shown you some of the functionality available. One word of caution before we wrap up: not everything is bindable, and sometimes the values you try to connect to are incompatible with the properties you are trying to bind. There is a lot of flexibility in binding, but it won't work in every situation.

One example of a problem not addressed by binding is when a control has no suitable properties to bind to, as with the detail page zoom feature. This zoom slider needs to interact with the `ScrollView` that displays the image, but `ScrollView` can only be updated through its `ChangeView` method. In this case, we use conventional event handlers to keep the `ScrollView` and the zoom slider in sync; see the `ZoomSlider_ValueChanged` and `MainImageScroll_ViewChanged` methods in `DetailPage` for details.

Nonetheless, binding is a powerful and flexible way to simplify your code and keep your UI logic separate from your data logic. This will make it much easier for you to adjust either side of this divide while reducing the risk of introducing bugs on the other side.

One example of UI and data separation is with the `ImageFileInfo.ImageTitle` property. This property (and the `ImageRating` property) is slightly different than the `ItemSize` property you created in Part 4 because the value is stored in the file metadata (exposed through the `ImageProperties` type) instead of in a field. Additionally, `ImageTitle` returns the `ImageName` value (set to the file name) if there is no title in the file metadata.